# Python Programming

I B.Tech. | Activity-Based Learning Assignment

10 Case Studies

## Activity-Based Learning

Programs, flowcharts, and example output

**Student Name:** **[YOUR NAME HERE]**

**Roll Number:** **[YOUR ROLL NUMBER HERE]**

**Branch:** Computer Science & Engineering

**Year:** I B.Tech.

# About this project

This report contains the 10 Python programs in this folder. They use sample data to show ideas like loops, conditions, strings, lists, and tuples. Each case has a short explanation, a flowchart, the code, and its example output.

## Case studies

| Case # | Title | Syllabus Concepts Demonstrated |
|---|---|---|
| **Case 02** | Progressive Electricity Billing and Anomaly Detection | Progressive Tariff Slabs, Fixed Fees, Surcharges, Rebates, Lists, Multi-Month Comparison |
| **Case 26** | Nested Loop Examination Seating Generator | 2D Nested Lists, Coordinate Offsets, Adjacency Conflict Prevention, Reserved Desks |
| **Case 27** | Advanced Numeric & Hollow Diamond Pattern Generator | Parity Validation, Nested Loops, Geometric Symmetry, Palindromic Multi-tier Numbers |
| **Case 31** | String-Based Examination Answer Evaluation | String Traversal, Slicing, Negative Marking, Bubble Sort Tie-Breaking, Item Analytics |
| **Case 42** | Student Marks Matrix Analysis & Performance Classification | 2D Nested Lists, Dual-Axis Matrix Reductions, Grade Assignments, Remedial Intervention Alerts |
| **Case 48** | Tuple-Based Employee Shift Conflict Detection | Immutable Tuples, Time Parsing (HH:MM to Mins), Interval Collision Logic, Rest Auditing |
| **Case 56** | Student Group Formation with Score Balancing | Serpentine (Snake Draft) Allocation Algorithm, Score Variance Minimization, Nested Rosters |
| **Case 75** | Integrated Student Record Consistency Checker | Alphanumeric Syntax Validation, Duplicate ID Detection, Mark/Attendance Ranges, Contradiction Trapping |
| **Case 91** | Advanced String-Based Registration Record Analyzer | Delimited Tokenization, Format Grammar, Calendar Validation, Categorized Defect Audit |
| **Case 100** | Comprehensive Smart Campus Management Simulation | Capstone Integration: Academics, 75% Attendance Gating, Event Caps, Venue Scheduling |

# Case 02: Electricity Bill

## What it does

This program calculates a monthly electricity bill from the number of units used. It also compares each month with the previous month and marks a large change in usage.

## Main steps

Calculate the energy charge using the unit slabs.
Add the fixed meter charge.
Add a surcharge or subtract a rebate when needed.
Compare the usage with the previous month and print the bill.

## Python topics

Variables, arithmetic, `if` statements, loops, lists, and tuples.

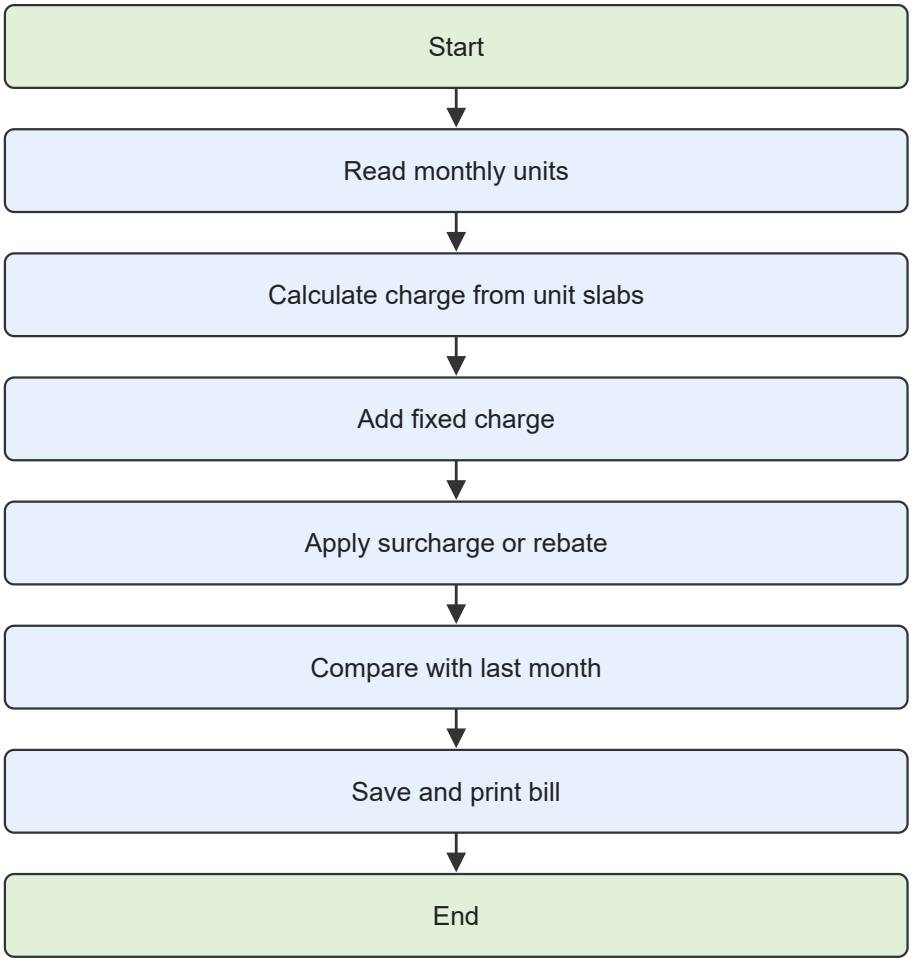
## Run it

```
python solution.py
```

The program uses sample monthly readings. See [output.txt](output.txt) for an example and [flowchart.svg](flowchart.svg) for the steps.

## Visual Program Flowchart Diagram

## Case 02: Electricity Bill




### Complete Python Source Code

```
"""Calculate monthly electricity bills and compare usage."""

def calculate_slab_energy_charge(units):
    """Calculate energy charges using progressive tariff slabs."""
    if units <= 0:
        return 0.0

    charge = 0.0
    # Slab 1: 0 - 100 units @ 2.50
    if units <= 100:
        charge = units * 2.50
    # Slab 2: 101 - 200 units @ 3.75
    elif units <= 200:
        charge = (100 * 2.50) + ((units - 100) * 3.75)
    # Slab 3: 201 - 400 units @ 5.20
    elif units <= 400:
        charge = (100 * 2.50) + (100 * 3.75) + ((units - 200) * 5.20)
    # Slab 4: Above 400 units @ 7.00
    else:
        charge = (100 * 2.50) + (100 * 3.75) + (200 * 5.20) + ((units - 400) * 7.00)
```

```
    return round(charge, 2)


def process_monthly_billing(month_records):
    """
    Process multi-month electricity consumption records.
    month_records is a list of tuples: (month_name, units_consumed)
    Returns a list of tuples with computed billing & anomaly alerts.
    """
    fixed_meter_charge = 75.00
    surcharge_threshold = 1500.00
    surcharge_rate = 0.05       # 5% if bill exceeds threshold
    rebate_threshold = 150.00   # units
    rebate_rate = 0.03          # 3% green rebate
    spike_threshold_pct = 30.0  # 30% jump compared to previous month

    detailed_bills = []

    for i in range(len(month_records)):
        month_name, units = month_records[i]

        # Energy charge based on progressive slabs
        energy_charge = calculate_slab_energy_charge(units)
        subtotal = energy_charge + fixed_meter_charge

        # Surcharge calculation
        surcharge = 0.0
        if subtotal > surcharge_threshold:
            surcharge = round(subtotal * surcharge_rate, 2)

        # Rebate calculation
        rebate = 0.0
        if units <= rebate_threshold:
            rebate = round((subtotal + surcharge) * rebate_rate, 2)

        net_payable = round(subtotal + surcharge - rebate, 2)

        # Anomaly detection against previous month
        anomaly_flag = "NORMAL"
        pct_change = 0.0
        if i > 0:
            prev_month, prev_units = month_records[i - 1]
            if prev_units > 0:
                pct_change = round(((units - prev_units) / prev_units) * 100, 2)
                if pct_change >= spike_threshold_pct:
                    anomaly_flag = f"SPIKE (+{pct_change}%)"
                elif pct_change <= -30.0:
                    anomaly_flag = f"DROP ({pct_change}%)"
            elif units > 0:
                anomaly_flag = "NEW CONSUMPTION"

        # Store detailed bill as tuple
        bill_entry = (
            month_name,
            units,
            energy_charge,
            fixed_meter_charge,
            surcharge,
            rebate,
            net_payable,
            anomaly_flag
        )
        detailed_bills.append(bill_entry)
```

```
    return detailed_bills


def display_billing_statement(consumer_info, bills):
    """Print an itemized and formatted monthly billing report."""
    consumer_id, consumer_name, meter_no = consumer_info

    separator = "=" * 88
    dash_line = "-" * 88

    print("\n" + separator)
    print(" " * 24 + "DR. RVR NRI POWER DISTRIBUTION CORP.")
    print(" " * 22 + "PROGRESSIVE ELECTRICITY BILLING STATEMENT")
    print(separator)
    print(f"Consumer ID : {consumer_id:<15} Consumer Name : {consumer_name}")
    print(f"Meter Number: {meter_no:<15} Tariff Class  : Domestic (LT-1)")
    print(dash_line)

    # Table Header
    print(f"{'Month':<10}{'Units':>7}{'Energy(Rs)':>12}{'Fixed(Rs)':>11}{'Surch(Rs)':>11}
{'Rebate(Rs)':>12}{'Net Pay(Rs)':>13}{'Status':>12}")
    print(dash_line)

    total_units = 0
    total_billed = 0.0
    anomalies = []

    # Iterate and unpack each bill tuple
    for entry in bills:
        month, units, energy, fixed, surch, reb, net, status = entry
        total_units += units
        total_billed += net

        if "SPIKE" in status:
            anomalies.append((month, units, status))

        print(f"{month:<10}{units:>7.1f}{energy:>12.2f}{fixed:>11.2f}{surch:>11.2f}{reb:>12.2f}
{net:>13.2f}  {status:<12}")

    print(dash_line)
    avg_units = round(total_units / len(bills), 2) if len(bills) > 0 else 0
    avg_bill = round(total_billed / len(bills), 2) if len(bills) > 0 else 0
    print(f"Total Consumption : {total_units:.1f} Units  | Average Monthly Consumption :
{avg_units:.1f} Units")
    print(f"Total Cumulative  : Rs {total_billed:,.2f} | Average Monthly Billed Amt  : Rs
{avg_bill:,.2f}")
    print(dash_line)

    if len(anomalies) > 0:
        print("[!] ANOMALY AUDIT REPORT:")
        for anom in anomalies:
            m, u, s = anom
            print(f"    - Month {m}: {u} units recorded -> Unusual surge detected: {s}")
        print("    Recommendation: Inspect meter wiring or audit high-power appliances for faults.")
    else:
        print("[*] Consumption profile is stable across all recorded billing cycles.")
    print(separator + "\n")


def run_demo():
    """Automated demonstration with verified test dataset."""
    consumer = ("CONS-50921", "Sri Durga Prasad", "MTR-AP-8841")
```

```
    # 6-Month consumption dataset demonstrating slabs, surcharges, rebates, and spikes
    monthly_data = [
        ("January", 90.0),    # Slab 1, eligible for green rebate
        ("February", 140.0),  # Slab 2, green rebate
        ("March", 210.0),     # Slab 3, moderate increase (+50%) -> Spike
        ("April", 460.0),     # Slab 4, surcharge applied (> Rs 1500) -> Spike (+119%)
        ("May", 480.0),       # High summer consumption
        ("June", 180.0)       # Drop post-summer
    ]

    bills = process_monthly_billing(monthly_data)
    display_billing_statement(consumer, bills)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================
                      DR. RVR NRI POWER DISTRIBUTION CORP.
                    PROGRESSIVE ELECTRICITY BILLING STATEMENT
================================================================================
Consumer ID : CONS-50921      Consumer Name : Sri Durga Prasad
Meter Number: MTR-AP-8841     Tariff Class  : Domestic (LT-1)
--------------------------------------------------------------------------------
Month       Units  Energy(Rs)  Fixed(Rs)  Surch(Rs)  Rebate(Rs)  Net Pay(Rs)      Status
--------------------------------------------------------------------------------
January      90.0      225.00      75.00       0.00        9.00       291.00  NORMAL
February    140.0      400.00      75.00       0.00       14.25       460.75  SPIKE (+55.56%)
March       210.0      677.00      75.00       0.00        0.00       752.00  SPIKE (+50.0%)
April       460.0     2085.00      75.00     108.00        0.00      2268.00  SPIKE (+119.05%)
May         480.0     2225.00      75.00     115.00        0.00      2415.00  NORMAL
June        180.0      550.00      75.00       0.00        0.00       625.00  DROP (-62.5%)
--------------------------------------------------------------------------------
Total Consumption : 1560.0 Units  | Average Monthly Consumption : 260.0 Units
Total Cumulative  : Rs 6,811.75 | Average Monthly Billed Amt  : Rs 1,135.29
--------------------------------------------------------------------------------
[!] ANOMALY AUDIT REPORT:
    - Month February: 140.0 units recorded -> Unusual surge detected: SPIKE (+55.56%)
    - Month March: 210.0 units recorded -> Unusual surge detected: SPIKE (+50.0%)
    - Month April: 460.0 units recorded -> Unusual surge detected: SPIKE (+119.05%)
    Recommendation: Inspect meter wiring or audit high-power appliances for faults.
================================================================================
```

# Case 26: Exam Seating Plan

## What it does

This program puts students into a row-and-column seating plan. It keeps reserved seats and tries not to place students from the same department next to each other.

## Main steps

Make a grid of vacant seats.
Mark the reserved seats.
Check nearby seats while placing each student.
Print the seating plan and show anyone left unseated.

## Python topics

Nested lists, tuples, nested loops, conditions, and string formatting.

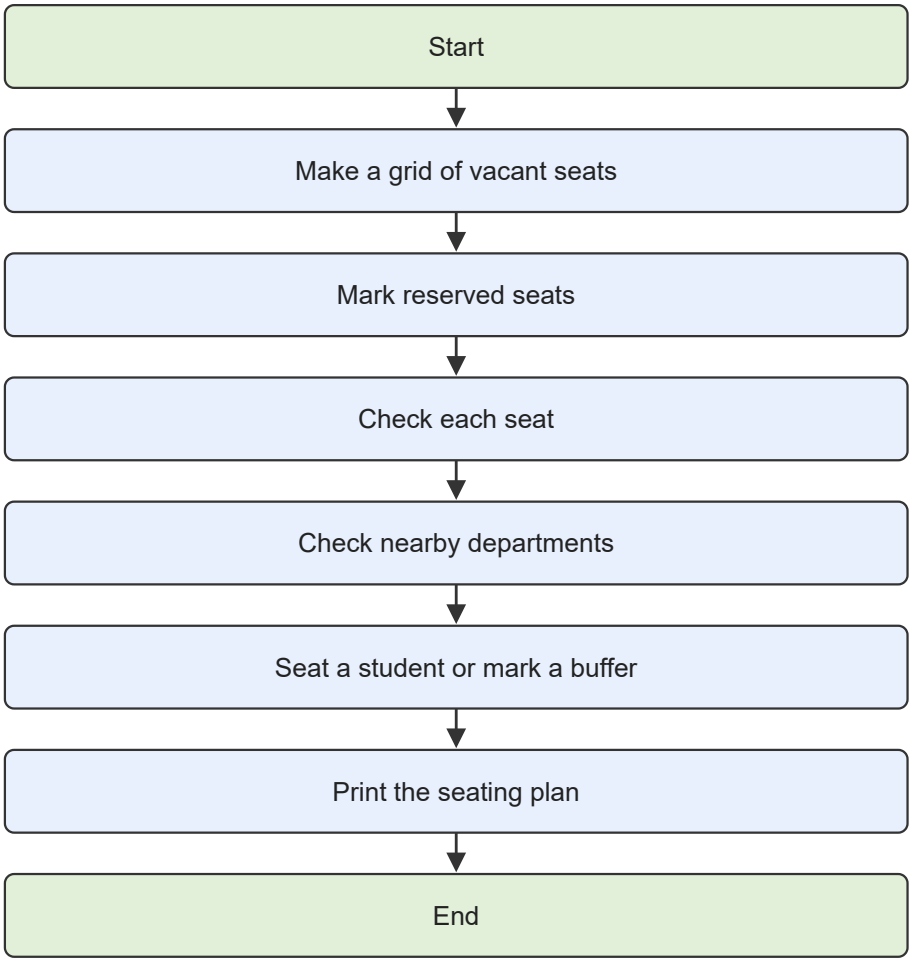
## Run it

```
python solution.py
```

The program uses sample students and room details. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

# Case 26: Exam Seating Plan




## Complete Python Source Code

```
"""Make a seating plan and check nearby seats."""

def create_hall_grid(rows, cols):
    """Initialize a 2D grid of size rows x cols with vacant desk indicators."""
    grid = []
    for r in range(rows):
        row_desks = []
        for c in range(cols):
            row_desks.append("[ VACANT ]")
        grid.append(row_desks)
    return grid


def is_adjacent_conflict(grid, r, c, dept, rows, cols):
    """
    Check if placing a student of department 'dept' at (r, c) causes
    an adjacent conflict (top, bottom, left, or right neighbor with same dept).
    """
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dr, dc in directions:
```

```
        nr = r + dr
        nc = c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            neighbor = grid[nr][nc]
            # Neighbor format: "[CSE-101]" -> dept is prefix before hyphen
            if "-" in neighbor:
                # Extract dept substring
                clean = neighbor.replace("[", "").replace("]", "").strip()
                n_dept = clean.split("-")[0]
                if n_dept == dept:
                    return True
    return False


def generate_seating_arrangement(rows, cols, student_pool, reserved_coords):
    """
    Allocates seats using nested loops, alternate branch restrictions,
    and reserved seat preservation.
    student_pool: list of tuples (roll_no, dept, name)
    reserved_coords: list of tuples (r, c, reserve_label)
    """
    grid = create_hall_grid(rows, cols)

    # 1. Mark reserved seats
    for res in reserved_coords:
        rr, rc, label = res
        if 0 <= rr < rows and 0 <= rc < cols:
            grid[rr][rc] = f"[{label:^8}]"

    allocated_count = 0
    unallocated_students = []

    # Create an active working queue/list of students
    pending_students = []
    for s in student_pool:
        pending_students.append(s)

    # 2. Iterate through grid using nested loops to allocate seats
    for r in range(rows):
        for c in range(cols):
            # Skip if seat is already reserved
            if grid[r][c] != "[ VACANT ]":
                continue

            placed = False
            # Search pending students for a department that doesn't violate adjacent constraints
            for idx in range(len(pending_students)):
                roll_no, dept, name = pending_students[idx]

                if not is_adjacent_conflict(grid, r, c, dept, rows, cols):
                    # Valid placement found
                    seat_label = f"[{dept[:3]}-{roll_no[-3:]}]"
                    grid[r][c] = seat_label
                    allocated_count += 1
                    pending_students.pop(idx)
                    placed = True
                    break

            if not placed and len(pending_students) > 0:
                # If no compatible student fits adjacency rule right now, leave desk vacant for
separation
                grid[r][c] = "[ BUFFER ]"

    unallocated_students = pending_students
```

```
    return grid, allocated_count, unallocated_students


def display_hall_layout(grid, rows, cols, hall_name, capacity, allocated, unallocated):
    """Render a clean ASCII map of the examination hall."""
    border = "=" * (cols * 14 + 14)
    print("\n" + border)
    print(f"{hall_name.center(cols * 14 + 14)}")
    print(f"{'EXAMINATION SEATING MATRIX & AUDIT REPORT'.center(cols * 14 + 14)}")
    print(border)

    # Column Header
    header = f"{'Row / Col':<10}"
    for c in range(cols):
        header += f"   Col-{c+1:<3}   "
    print(header)
    print("-" * (cols * 14 + 14))

    # Print 2D Grid using nested loops
    for r in range(rows):
        row_str = f"Row-{r+1:<5} |"
        for c in range(cols):
            row_str += f" {grid[r][c]:^10} |"
        print(row_str)

    print("-" * (cols * 14 + 14))
    print(f"Hall Capacity      : {capacity} Desks ({rows} Rows x {cols} Columns)")
    print(f"Students Allocated : {allocated}")
    print(f"Desks Vacant/Buffer: {capacity - allocated}")
    print(f"Unallocated Count  : {len(unallocated)}")

    if len(unallocated) > 0:
        print("\n[!] WARNING: Unallocated Students (Capacity/Adjacency overflow):")
        for u in unallocated:
            roll, dept, name = u
            print(f"    - Roll: {roll} | Dept: {dept:<5} | Name: {name}")
    else:
        print("\n[*] SUCCESS: All students allocated adhering to alternate-branch constraints!")
    print(border + "\n")


def run_demo():
    """Execute seating generation demo for a 4x4 examination hall."""
    rows = 4
    cols = 4
    capacity = rows * cols

    # Reserved seats (row_idx, col_idx, reason_label)
    reserved = [
        (0, 0, "PWD-SEAT"),   # Physically challenged reserved desk
        (3, 3, "INVIG-BF")    # Invigilator reserve buffer
    ]

    # Mixed student candidates across CSE, ECE, MECH, and IT
    students = [
        ("26CSE101", "CSE", "Aarav Sharma"),
        ("26ECE201", "ECE", "Bhavna Patel"),
        ("26CSE102", "CSE", "Chaitanya Rao"),
        ("26ECE202", "ECE", "Deepa Verma"),
        ("26MEC301", "MEC", "Eashwar Reddy"),
        ("26INF401", "INF", "Farhan Akhtar"),
        ("26CSE103", "CSE", "Gautam Krishna"),
        ("26ECE203", "ECE", "Harika Devi"),
        ("26MEC302", "MEC", "Indrajit Paul"),
```

```
        ("26INF402", "INF", "Jaya Lakshmi"),
        ("26CSE104", "CSE", "Kalyan Varma"),
        ("26ECE204", "ECE", "Lavanya Nair"),
        ("26MEC303", "MEC", "Manoj Kumar"),
        ("26INF403", "INF", "Nandini Sen")
    ]

    grid, allocated, unallocated = generate_seating_arrangement(rows, cols, students, reserved)
    display_hall_layout(grid, rows, cols, "ROOM 302 - MAIN ACADEMIC BLOCK", capacity, allocated,
unallocated)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
======================================================================
                    ROOM 302 - MAIN ACADEMIC BLOCK
              EXAMINATION SEATING MATRIX & AUDIT REPORT
======================================================================
Row / Col    Col-1        Col-2        Col-3        Col-4
----------------------------------------------------------------------
Row-1     | [PWD-SEAT] | [CSE-101]  | [ECE-201]  | [CSE-102]  |
Row-2     | [ECE-202]  | [MEC-301]  | [INF-401]  | [ECE-203]  |
Row-3     | [CSE-103]  | [INF-402]  | [MEC-302]  | [CSE-104]  |
Row-4     | [ECE-204]  | [MEC-303]  | [INF-403]  | [INVIG-BF] |
----------------------------------------------------------------------
Hall Capacity      : 16 Desks (4 Rows x 4 Columns)
Students Allocated : 14
Desks Vacant/Buffer: 2
Unallocated Count  : 0

[*] SUCCESS: All students allocated adhering to alternate-branch constraints!
======================================================================
```

# Case 27: Number Patterns

## What it does

This program checks a pattern size and prints two number diamonds. The size must be an odd number from 3 to 25.

## Main steps

Check that the size is allowed.
Use loops to print the top half of each diamond.
Print the bottom half in reverse order.

## Python topics

Conditions, `range()`, nested loops, strings, and pattern printing.

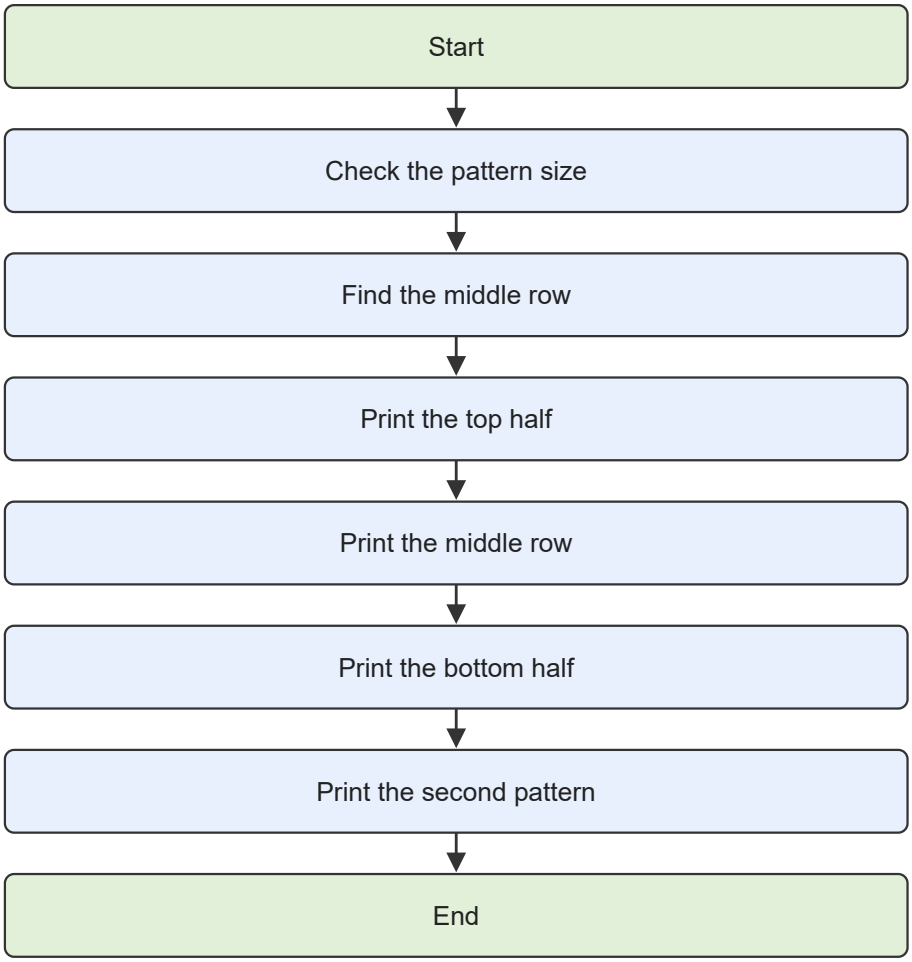
## Run it

```
python solution.py
```

The program tries a few sizes, including an invalid one. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

## Case 27: Number Patterns




### Complete Python Source Code

```
"""Check a pattern size and print two number diamonds."""

def validate_pattern_dimension(n):
    """
    Validates that pattern dimension n is a positive odd integer
    within acceptable display limits (3 to 25).
    Returns (is_valid, error_message).
    """
    if not isinstance(n, int):
        return False, "Dimension must be an integer value."
    if n < 3:
        return False, "Dimension is too small; minimum size is 3."
    if n > 25:
        return False, "Dimension exceeds terminal display limit (maximum 25)."
    if n % 2 == 0:
        return False, f"Dimension {n} is even. A symmetric hollow diamond requires an ODD dimension."
    return True, "Dimension is valid."


def generate_hollow_numeric_diamond(n):
```

```
    """
    Generates a hollow diamond of numbers of height n (n must be odd).
    Outer borders display the current layer depth number with symmetric spacing.
    Returns list of pattern lines.
    """
    lines = []
    mid = n // 2  # 0 to mid is upper half, mid+1 to n-1 is lower half

    # 1. Upper Half (including center row)
    for i in range(mid + 1):
        outer_spaces = " " * (mid - i)
        num_char = str(i + 1)

        if i == 0:
            line = outer_spaces + num_char
        else:
            inner_spaces = " " * (2 * i - 1)
            line = outer_spaces + num_char + inner_spaces + num_char

        lines.append(line)

    # 2. Lower Half (symmetric reflection)
    for i in range(mid - 1, -1, -1):
        outer_spaces = " " * (mid - i)
        num_char = str(i + 1)

        if i == 0:
            line = outer_spaces + num_char
        else:
            inner_spaces = " " * (2 * i - 1)
            line = outer_spaces + num_char + inner_spaces + num_char

        lines.append(line)

    return lines


def generate_symmetric_palindromic_diamond(n):
    """
    Generates a full palindromic diamond where each row is a palindrome
    1 2 ... k ... 2 1 with proportional outer indentations.
    """
    lines = []
    mid = n // 2

    # Upper Half + Center
    for i in range(mid + 1):
        outer_spaces = "  " * (mid - i)
        # Ascending sequence
        row_nums = []
        for val in range(1, i + 2):
            row_nums.append(str(val))
        # Descending sequence
        for val in range(i, 0, -1):
            row_nums.append(str(val))

        line = outer_spaces + " ".join(row_nums)
        lines.append(line)

    # Lower Half
    for i in range(mid - 1, -1, -1):
        outer_spaces = "  " * (mid - i)
        row_nums = []
        for val in range(1, i + 2):
```

```
            row_nums.append(str(val))
        for val in range(i, 0, -1):
            row_nums.append(str(val))

        line = outer_spaces + " ".join(row_nums)
        lines.append(line)

    return lines


def display_pattern_suite(dimension):
    """Validates input, generates both patterns, and renders formatted output."""
    print("=" * 65)
    print("       ADVANCED NUMERIC & HOLLOW DIAMOND PATTERN SUITE       ")
    print("=" * 65)
    print(f"Target Dimension Request : N = {dimension}")

    is_valid, msg = validate_pattern_dimension(dimension)
    if not is_valid:
        print(f"[ERROR] Input Validation Failed: {msg}")
        print("=" * 65 + "\n")
        return

    print(f"[STATUS] Validation Passed: {msg}\n")

    print("--- Pattern 1: Symmetric Hollow Numeric Diamond ---")
    hollow_lines = generate_hollow_numeric_diamond(dimension)
    for line in hollow_lines:
        print(line)

    print("\n--- Pattern 2: Multi-Layer Palindromic Number Diamond ---")
    palindromic_lines = generate_symmetric_palindromic_diamond(dimension)
    for line in palindromic_lines:
        print(line)

    print("=" * 65 + "\n")


def run_demo():
    """Run test cases including invalid boundary tests and successful renders."""
    # Test 1: Even dimension rejection
    display_pattern_suite(6)

    # Test 2: Standard Odd dimension N = 7
    display_pattern_suite(7)

    # Test 3: Complex Odd dimension N = 9
    display_pattern_suite(9)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
=================================================================
       ADVANCED NUMERIC & HOLLOW DIAMOND PATTERN SUITE
=================================================================
Target Dimension Request : N = 6
[ERROR] Input Validation Failed: Dimension 6 is even. A symmetric hollow diamond requires an ODD
dimension.
=================================================================
```

```
======================================================================
       ADVANCED NUMERIC & HOLLOW DIAMOND PATTERN SUITE
======================================================================
Target Dimension Request : N = 7
[STATUS] Validation Passed: Dimension is valid.

--- Pattern 1: Symmetric Hollow Numeric Diamond ---
   1
  2 2
 3   3
4     4
 3   3
  2 2
   1

--- Pattern 2: Multi-Layer Palindromic Number Diamond ---
      1
    1 2 1
  1 2 3 2 1
1 2 3 4 3 2 1
  1 2 3 2 1
    1 2 1
      1
======================================================================

======================================================================
       ADVANCED NUMERIC & HOLLOW DIAMOND PATTERN SUITE
======================================================================
Target Dimension Request : N = 9
[STATUS] Validation Passed: Dimension is valid.

--- Pattern 1: Symmetric Hollow Numeric Diamond ---
    1
   2 2
  3   3
 4     4
5       5
 4     4
  3   3
   2 2
    1

--- Pattern 2: Multi-Layer Palindromic Number Diamond ---
        1
      1 2 1
    1 2 3 2 1
  1 2 3 4 3 2 1
1 2 3 4 5 4 3 2 1
  1 2 3 4 3 2 1
    1 2 3 2 1
      1 2 1
        1
======================================================================
```

# Case 31: Answer Key

## What it does

This program compares students' answers with an answer key. It gives marks for correct answers, applies negative marks for wrong answers, and shows the results.

## Main steps

Check that the answer strings have the right format.
Compare each answer with the key.
Calculate scores and sort the student results.
Print the scores and question information.

## Python topics

Strings, indexing, loops, lists, conditions, and sorting with loops.

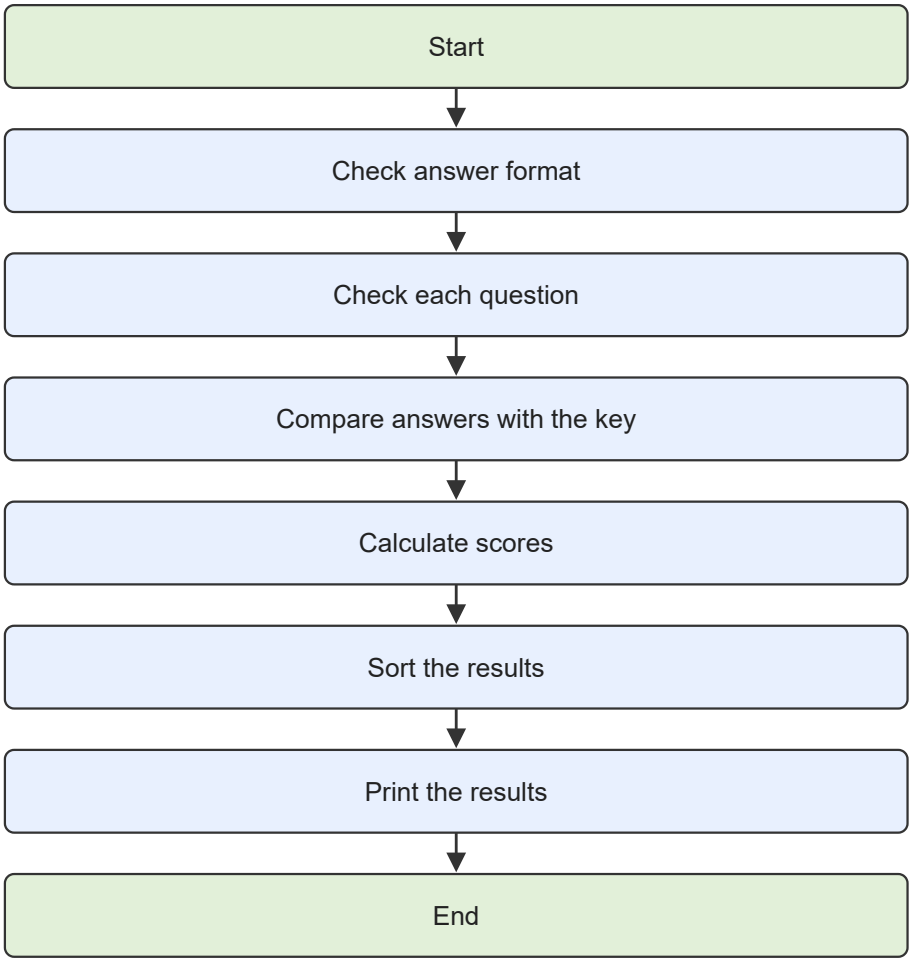
## Run it

```
python solution.py
```

The program uses sample answers. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

## Case 31: Answer Key




### Complete Python Source Code

```
"""Compare student answers with an answer key and calculate scores."""

def validate_answer_string(answer_str, expected_length):
    """
    Validates that student answer string matches expected length
    and contains only allowed characters ('A', 'B', 'C', 'D', 'X' for unattempted).
    """
    if len(answer_str) != expected_length:
        return False, f"Length mismatch: Expected {expected_length}, found {len(answer_str)}"

    valid_options = "ABCDX"
    for char in answer_str.upper():
        if char not in valid_options:
            return False, f"Invalid character detected: '{char}'. Only A, B, C, D, X permitted."

    return True, "Valid"


def evaluate_single_student(roll_no, student_name, student_answers, answer_key):
    """
```

```
    Evaluates a single student's answers against the master key.
    Marking scheme: +4 for correct, -1 for incorrect, 0 for unattempted ('X').
    Returns a result tuple.
    """
    answers = student_answers.upper()
    total_q = len(answer_key)
    correct = 0
    incorrect = 0
    unattempted = 0
    question_status = []  # List of 'C', 'I', 'U'

    for i in range(total_q):
        s_ans = answers[i]
        k_ans = answer_key[i]

        if s_ans == 'X':
            unattempted += 1
            question_status.append('U')
        elif s_ans == k_ans:
            correct += 1
            question_status.append('C')
        else:
            incorrect += 1
            question_status.append('I')

    max_marks = total_q * 4
    net_score = (correct * 4) - (incorrect * 1)
    attempted = correct + incorrect
    accuracy_pct = round((correct / attempted * 100), 2) if attempted > 0 else 0.0
    net_pct = round((net_score / max_marks * 100), 2)

    return (
        roll_no,
        student_name,
        correct,
        incorrect,
        unattempted,
        net_score,
        accuracy_pct,
        net_pct,
        question_status
    )


def sort_candidates_by_performance(results):
    """
    Sorts results in descending order of net_score.
    Tie-breaking rule: candidate with fewer negative marks (incorrect) ranks higher.
    Implemented purely using bubble sort to adhere strictly to syllabus.
    """
    n = len(results)
    sorted_list = []
    for item in results:
        sorted_list.append(item)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            # Unpack tuples to compare
            r1_score = sorted_list[j][5]
            r1_incorrect = sorted_list[j][3]

            r2_score = sorted_list[j + 1][5]
            r2_incorrect = sorted_list[j + 1][3]
```

```
            # Condition to swap: lower score, or same score with MORE incorrect marks
            swap = False
            if r1_score < r2_score:
                swap = True
            elif r1_score == r2_score and r1_incorrect > r2_incorrect:
                swap = True

            if swap:
                temp = sorted_list[j]
                sorted_list[j] = sorted_list[j + 1]
                sorted_list[j + 1] = temp

    return sorted_list


def compute_question_analytics(evaluated_batch, total_questions):
    """
    Analyzes item difficulty: determines correct, incorrect, and skip count
    for each question across the student cohort.
    """
    analytics = []
    num_students = len(evaluated_batch)

    for q_idx in range(total_questions):
        c_count = 0
        i_count = 0
        u_count = 0
        for student in evaluated_batch:
            status_list = student[8]
            st = status_list[q_idx]
            if st == 'C':
                c_count += 1
            elif st == 'I':
                i_count += 1
            else:
                u_count += 1
        success_rate = round((c_count / num_students) * 100, 1) if num_students > 0 else 0.0
        analytics.append((q_idx + 1, c_count, i_count, u_count, success_rate))

    return analytics


def display_examination_report(answer_key, batch_records):
    """Displays full evaluation, ranked merit list, and question analytics."""
    total_q = len(answer_key)
    max_marks = total_q * 4
    evaluated_batch = []
    validation_errors = []

    # Process each student record
    for rec in batch_records:
        roll, name, ans_str = rec
        is_valid, msg = validate_answer_string(ans_str, total_q)
        if not is_valid:
            validation_errors.append((roll, name, msg))
            continue
        eval_tuple = evaluate_single_student(roll, name, ans_str, answer_key)
        evaluated_batch.append(eval_tuple)

    ranked_results = sort_candidates_by_performance(evaluated_batch)

    border = "=" * 92
    dash = "-" * 92
```

```
    print("\n" + border)
    print(" " * 22 + "NATIONAL ENTRANCE EXAMINATION SYSTEM")
    print(" " * 20 + "AUTOMATED ANSWER EVALUATION & MERIT AUDIT")
    print(border)
    print(f"Master Answer Key ({total_q} Questions): {answer_key}")
    print(f"Marking Scheme: Correct: +4 | Incorrect: -1 | Unanswered: 0 | Max Marks: {max_marks}")
    print(dash)

    # Print Ranked Merit Table
    print(f"{'Rank':<5}{'Roll No':<11}{'Student Name':<18}{'Correct':>8}{'Wrong':>7}{'Skip':>6}{'Net
Score':>11}{'Accuracy':>10}{'Net %':>8}{'Result':>8}")
    print(dash)

    rank = 1
    for row in ranked_results:
        roll, name, c, w, u, net, acc, net_p, _ = row
        status = "QUALIFIED" if net >= (max_marks * 0.40) else "NEEDS IMPR"
        print(f"{rank:<5}{roll:<11}{name:<18}{c:>8}{w:>7}{u:>6}{net:>11}/{max_marks}{acc:>9.1f}%
{net_p:>7.1f}%  {status}")
        rank += 1

    print(dash)

    if len(validation_errors) > 0:
        print("[!] DISQUALIFIED / MALFORMED ANSWER STRINGS DETECTED:")
        for err in validation_errors:
            eroll, ename, emsg = err
            print(f"    - Roll: {eroll} | Name: {ename} -> Reason: {emsg}")
        print(dash)

    # Question Analytics
    q_analytics = compute_question_analytics(evaluated_batch, total_q)
    print("\n[*] QUESTION-WISE DIFFICULTY & SUCCESS RATE ANALYTICS:")
    print(f"{'Q No':<6}{'Correct Count':>15}{'Wrong Count':>13}{'Skipped':>10}{'Success Rate':>15}
{'Difficulty Classification':>28}")
    print("-" * 88)

    for qa in q_analytics:
        q_num, cc, wc, sc, s_rate = qa
        diff = "EASY" if s_rate >= 75.0 else ("MODERATE" if s_rate >= 40.0 else "HARD / FREQUENTLY
MISSED")
        print(f"Q-{q_num:<4}{cc:>15}{wc:>13}{sc:>10}{s_rate:>14.1f}%  {diff:>26}")

    print(border + "\n")


def run_demo():
    """Run test cases with realistic entrance exam answer sheets."""
    # Official Master Answer Key for 15 questions
    official_key = "ABCDABCDABCDABC"

    students = [
        ("26ENG001", "Kavya Reddy",    "ABCDABCDABCDABC"),  # 100% perfect
        ("26ENG002", "Manoj Varma",    "ABCDABCDABCDXXX"),  # 12 attempted, all correct
        ("26ENG003", "Naveen Kumar",   "ABCDABCDABCDABA"),  # 1 wrong at end
        ("26ENG004", "Pooja Sharma",   "A BCDABCDABCDABC"), # Length 16 (error!)
        ("26ENG005", "Rajesh Gupta",   "ABCDABCDABCDABZ"),  # Invalid char 'Z' (error!)
        ("26ENG006", "Sneha Patel",    "BACDBACDBACDBAC"),  # High error rate
        ("26ENG007", "Teja Vardhan",   "ABCDABCDABCDABA")   # Tied with Naveen Kumar
    ]

    display_examination_report(official_key, students)
```

```
if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================================
                      NATIONAL ENTRANCE EXAMINATION SYSTEM
                    AUTOMATED ANSWER EVALUATION & MERIT AUDIT
================================================================================================
Master Answer Key (15 Questions): ABCDABCDABCDABC
Marking Scheme: Correct: +4 | Incorrect: -1 | Unanswered: 0 | Max Marks: 60
------------------------------------------------------------------------------------------------
Rank Roll No    Student Name       Correct  Wrong  Skip  Net Score  Accuracy   Net %  Result
------------------------------------------------------------------------------------------------
1    26ENG001   Kavya Reddy             15      0     0         60/60    100.0%  100.0%  QUALIFIED
2    26ENG003   Naveen Kumar            14      1     0         55/60     93.3%   91.7%  QUALIFIED
3    26ENG007   Teja Vardhan            14      1     0         55/60     93.3%   91.7%  QUALIFIED
4    26ENG002   Manoj Varma             12      0     3         48/60    100.0%   80.0%  QUALIFIED
5    26ENG006   Sneha Patel              7      8     0         20/60     46.7%   33.3%  NEEDS IMPR
------------------------------------------------------------------------------------------------
[!] DISQUALIFIED / MALFORMED ANSWER STRINGS DETECTED:
    - Roll: 26ENG004 | Name: Pooja Sharma -> Reason: Length mismatch: Expected 15, found 16
    - Roll: 26ENG005 | Name: Rajesh Gupta -> Reason: Invalid character detected: 'Z'. Only A, B, C, D,
X permitted.
------------------------------------------------------------------------------------------------

[*] QUESTION-WISE DIFFICULTY & SUCCESS RATE ANALYTICS:
Q No    Correct Count  Wrong Count   Skipped   Success Rate   Difficulty Classification
------------------------------------------------------------------------------------------
Q-1                 4            1         0          80.0%                       EASY
Q-2                 4            1         0          80.0%                       EASY
Q-3                 5            0         0         100.0%                       EASY
Q-4                 5            0         0         100.0%                       EASY
Q-5                 4            1         0          80.0%                       EASY
Q-6                 4            1         0          80.0%                       EASY
Q-7                 5            0         0         100.0%                       EASY
Q-8                 5            0         0         100.0%                       EASY
Q-9                 4            1         0          80.0%                       EASY
Q-10                4            1         0          80.0%                       EASY
Q-11                5            0         0         100.0%                       EASY
Q-12                5            0         0         100.0%                       EASY
Q-13                3            1         1          60.0%                   MODERATE
Q-14                3            1         1          60.0%                   MODERATE
Q-15                2            2         1          40.0%                   MODERATE
================================================================================================
```

# Case 42: Marks Analysis

## What it does

This program stores marks in a two-dimensional list. It works out each student's total and grade, then checks how the class did in each subject.

## Main steps

Read the marks from the sample matrix.
Use each row to find a student's total and grade.
Use each column to find subject results.
Print the results and any subjects that may need more help.

## Python topics

Lists inside lists, loops, totals, averages, tuples, and `if` statements.

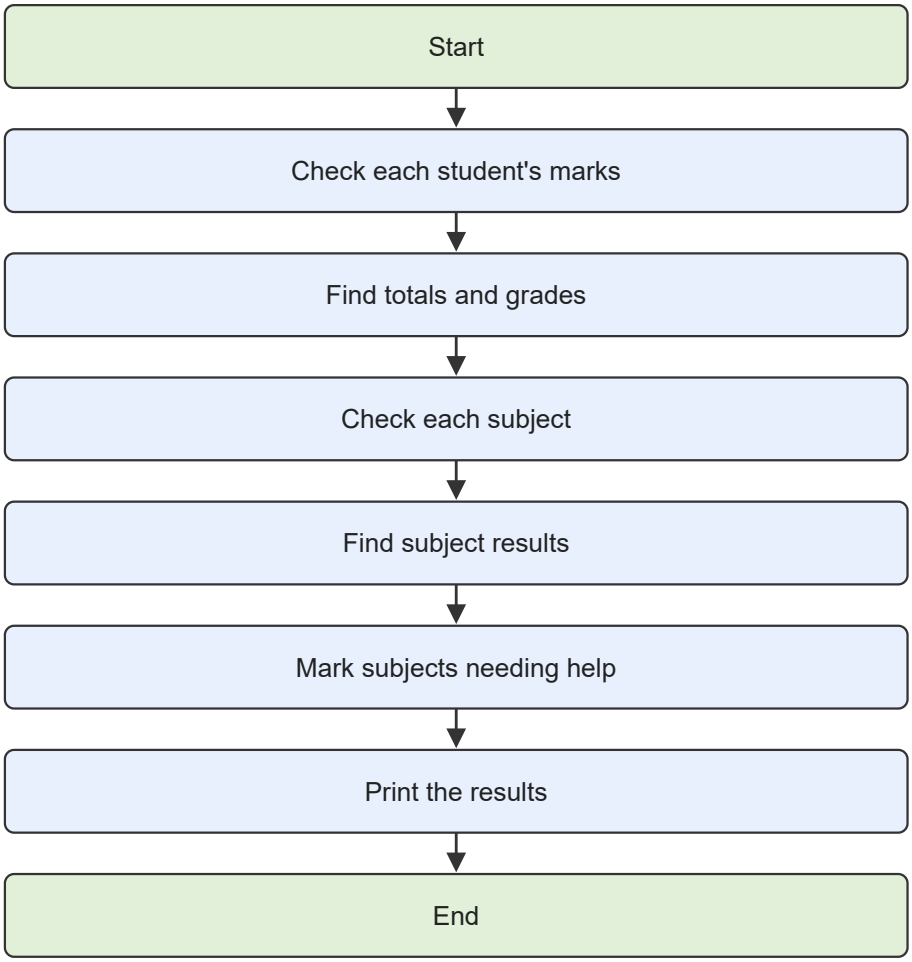
## Run it

```
python solution.py
```

See [output.txt](output.txt) for the printed report and [flowchart.svg](flowchart.svg) for the main steps.

## Visual Program Flowchart Diagram

# Case 42: Marks Analysis




## Complete Python Source Code

```
"""Calculate student grades and check subject results."""

def compute_grade(percentage, has_failed_any_subject):
    """Determine letter grade according to academic grading scale."""
    if has_failed_any_subject or percentage < 40.0:
        return "F"
    elif percentage >= 90.0:
        return "O"
    elif percentage >= 80.0:
        return "A+"
    elif percentage >= 70.0:
        return "A"
    elif percentage >= 60.0:
        return "B+"
    elif percentage >= 50.0:
        return "B"
    else:
        return "C"
```

```
def analyze_student_matrix(students_info, subjects, marks_matrix):
    """
    Analyzes 2D marks matrix (nested lists).
    students_info: list of tuples (roll_no, student_name)
    subjects: list of subject names (e.g., ['Math', 'Physics', 'Python', 'Electronics'])
    marks_matrix: nested list where marks_matrix[i][j] is mark of student i in subject j
    """
    num_students = len(students_info)
    num_subjects = len(subjects)
    pass_mark_per_subject = 40.0

    # 1. Row-Wise Analysis (Student Performance)
    student_summaries = []

    for i in range(num_students):
        roll_no, name = students_info[i]
        marks_row = marks_matrix[i]

        row_total = 0
        failed_count = 0
        min_mark = 101.0
        max_mark = -1.0

        for mark in marks_row:
            row_total += mark
            if mark < pass_mark_per_subject:
                failed_count += 1
            if mark < min_mark:
                min_mark = mark
            if mark > max_mark:
                max_mark = mark

        percentage = round((row_total / (num_subjects * 100)) * 100, 2)
        has_failed = failed_count > 0
        grade = compute_grade(percentage, has_failed)
        status = "PASSED" if not has_failed else f"FAILED ({failed_count} Sub)"

        student_summaries.append((
            roll_no,
            name,
            marks_row,
            row_total,
            percentage,
            grade,
            status
        ))

    # 2. Column-Wise Analysis (Subject Performance)
    subject_summaries = []

    for j in range(num_subjects):
        col_total = 0
        col_max = -1.0
        col_min = 101.0
        pass_count = 0

        for i in range(num_students):
            m = marks_matrix[i][j]
            col_total += m
            if m >= pass_mark_per_subject:
                pass_count += 1
            if m > col_max:
                col_max = m
            if m < col_min:
```

```
                col_min = m

        sub_avg = round(col_total / num_students, 2) if num_students > 0 else 0.0
        sub_pass_pct = round((pass_count / num_students) * 100, 2) if num_students > 0 else 0.0
        needs_attention = (sub_avg < 60.0) or (sub_pass_pct < 75.0)

        subject_summaries.append((
            subjects[j],
            sub_avg,
            col_max,
            col_min,
            sub_pass_pct,
            needs_attention
        ))

    return student_summaries, subject_summaries


def rank_students(student_summaries):
    """Rank students based on total marks using bubble sort."""
    ranked = []
    for item in student_summaries:
        ranked.append(item)

    n = len(ranked)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if ranked[j][3] < ranked[j + 1][3]:  # Compare total marks
                temp = ranked[j]
                ranked[j] = ranked[j + 1]
                ranked[j + 1] = temp

    return ranked


def display_matrix_report(subjects, student_summaries, subject_summaries):
    """Render a comprehensive academic scorecard and diagnostic report."""
    border = "=" * 98
    dash = "-" * 98

    print("\n" + border)
    print(" " * 26 + "DR. RVR NRI INSTITUTE OF TECHNOLOGY")
    print(" " * 24 + "DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING")
    print(" " * 23 + "SEMESTER MARKS MATRIX & PERFORMANCE AUDIT")
    print(border)

    # Table Header with Dynamic Subject Columns
    header = f"{'Roll No':<10}{'Student Name':<16}"
    for sub in subjects:
        header += f"{sub[:6]:>8}"
    header += f"{'Total':>9}{'Pct %':>8}{'Grade':>7}{'Status':>16}"
    print(header)
    print(dash)

    # Print Student Rows
    for s in student_summaries:
        roll, name, marks, total, pct, grade, status = s
        row_str = f"{roll:<10}{name:<16}"
        for m in marks:
            row_str += f"{m:>8.1f}"
        row_str += f"{total:>9.1f}{pct:>7.1f}%{grade:>6}  {status:<16}"
        print(row_str)

    print(dash)
```

```
    # Column-Wise Summary Footer
    avg_row = f"{'SUBJECT AVG':<26}"
    for sub_stat in subject_summaries:
        s_name, s_avg, _, _, _, _ = sub_stat
        avg_row += f"{s_avg:>8.1f}"
    print(avg_row)

    pass_pct_row = f"{'SUBJECT PASS %':<26}"
    for sub_stat in subject_summaries:
        _, _, _, _, s_pass, _ = sub_stat
        pass_pct_row += f"{s_pass:>7.1f}%"
    print(pass_pct_row)
    print(dash)

    # Top Performers (Merit List)
    ranked = rank_students(student_summaries)
    print("\n[*] TOP ACADEMIC PERFORMERS (MERIT RANKING):")
    top_limit = 3 if len(ranked) >= 3 else len(ranked)
    for r in range(top_limit):
        roll, name, _, tot, pct, gr, _ = ranked[r]
        print(f"    Rank #{r + 1}: {name} ({roll}) | Total: {tot:.1f} / {len(subjects)*100} ({pct}%) |
Grade: {gr}")

    print("\n[!] CURRICULUM INTERVENTION ALERTS (LOW-PERFORMING SUBJECTS):")
    flagged = False
    for sub_stat in subject_summaries:
        s_name, s_avg, s_max, s_min, s_pass, needs_att = sub_stat
        if needs_att:
            flagged = True
            print(f"    - Subject: {s_name} | Class Average: {s_avg:.1f} | Pass Rate: {s_pass:.1f}%")
            print(f"      Root Cause: Class average below 60.0% or pass rate below 75.0%. Remedial
sessions recommended.")
    if not flagged:
        print("    All subjects met institutional performance thresholds.")
    print(border + "\n")


def run_demo():
    """Run demonstration with 6 students across 5 subjects."""
    subjects = ["Maths", "Physics", "Python", "DataStr", "Digital"]

    students_info = [
        ("26CSE01", "Aaditya Roy"),
        ("26CSE02", "Bhavya Sri"),
        ("26CSE03", "Charan Teja"),
        ("26CSE04", "Divya Reddy"),
        ("26CSE05", "Eshwar Rao"),
        ("26CSE06", "Fatima Zahra")
    ]

    # 6 x 5 Nested List Marks Matrix
    marks_matrix = [
        [94.0, 88.0, 96.0, 91.0, 89.0],  # Top student
        [78.0, 72.0, 85.0, 68.0, 74.0],  # Strong student
        [32.0, 54.0, 62.0, 38.0, 50.0],  # Failed in Maths (32) and DataStr (38)
        [88.0, 82.0, 90.0, 84.0, 86.0],  # High performer
        [45.0, 48.0, 55.0, 42.0, 51.0],  # Just passed all
        [65.0, 58.0, 72.0, 35.0, 60.0]   # Failed in DataStr (35)
    ]

    student_summaries, subject_summaries = analyze_student_matrix(students_info, subjects,
marks_matrix)
    display_matrix_report(subjects, student_summaries, subject_summaries)
```

```
if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================================
                        DR. RVR NRI INSTITUTE OF TECHNOLOGY
                     DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
                    SEMESTER MARKS MATRIX & PERFORMANCE AUDIT
================================================================================================
Roll No   Student Name       Maths  Physic  Python  DataSt  Digita    Total   Pct %  Grade
Status
------------------------------------------------------------------------------------------------
26CSE01   Aaditya Roy         94.0    88.0    96.0    91.0    89.0    458.0   91.6%     O  PASSED
26CSE02   Bhavya Sri          78.0    72.0    85.0    68.0    74.0    377.0   75.4%     A  PASSED
26CSE03   Charan Teja         32.0    54.0    62.0    38.0    50.0    236.0   47.2%     F  FAILED (2
Sub)
26CSE04   Divya Reddy         88.0    82.0    90.0    84.0    86.0    430.0   86.0%    A+  PASSED
26CSE05   Eshwar Rao          45.0    48.0    55.0    42.0    51.0    241.0   48.2%     C  PASSED
26CSE06   Fatima Zahra        65.0    58.0    72.0    35.0    60.0    290.0   58.0%     F  FAILED (1
Sub)
------------------------------------------------------------------------------------------------
SUBJECT AVG                   67.0    67.0    76.7    59.7    68.3
SUBJECT PASS %               83.3%  100.0%  100.0%   66.7%  100.0%
------------------------------------------------------------------------------------------------

[*] TOP ACADEMIC PERFORMERS (MERIT RANKING):
    Rank #1: Aaditya Roy (26CSE01) | Total: 458.0 / 500 (91.6%) | Grade: O
    Rank #2: Divya Reddy (26CSE04) | Total: 430.0 / 500 (86.0%) | Grade: A+
    Rank #3: Bhavya Sri (26CSE02) | Total: 377.0 / 500 (75.4%) | Grade: A

[!] CURRICULUM INTERVENTION ALERTS (LOW-PERFORMING SUBJECTS):
    - Subject: DataStr | Class Average: 59.7 | Pass Rate: 66.7%
      Root Cause: Class average below 60.0% or pass rate below 75.0%. Remedial sessions recommended.
================================================================================================
```

# Case 48: Shift Checker

## What it does

This program checks a list of employee shifts. It finds shifts that overlap and checks whether there is enough rest time between shifts.

## Main steps

Convert each time from hours and minutes to minutes.
Check each pair of shifts.
Look for overlaps, long shifts, and short rest periods.
Print the shift list and any problems found.

## Python topics

Tuples, strings, lists, loops, comparisons, and arithmetic.

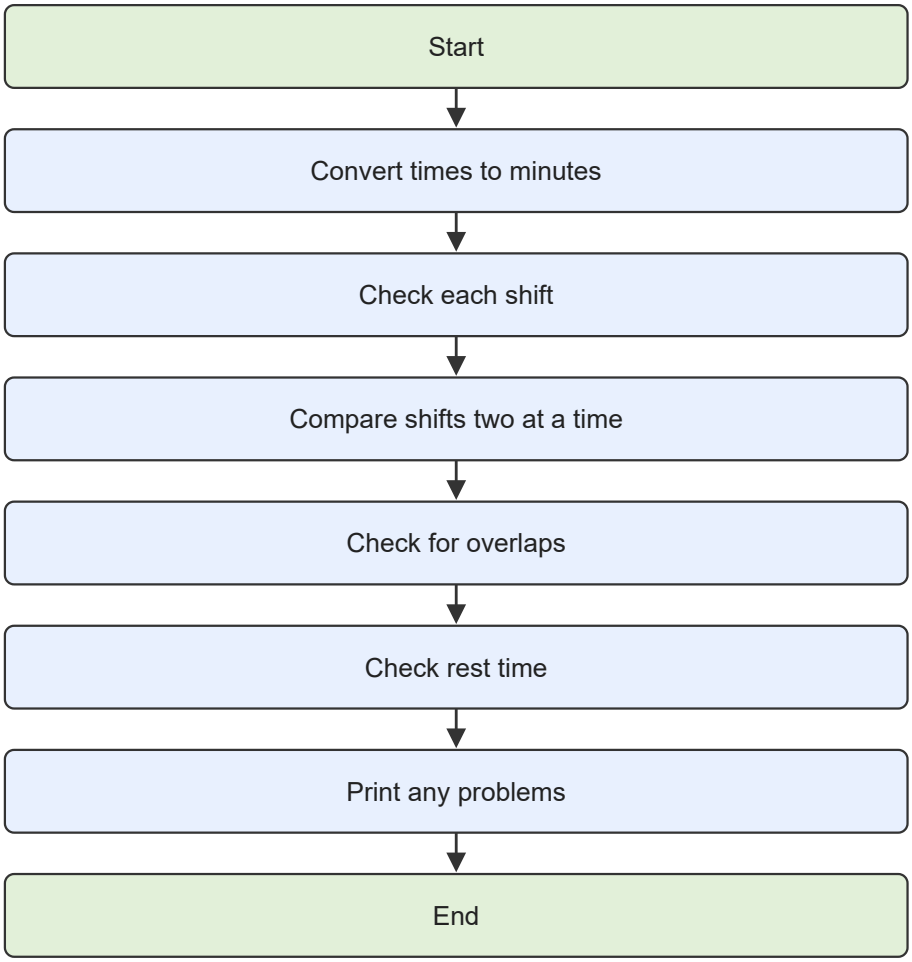
## Run it

```
python solution.py
```

The program uses sample shifts. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

# Case 48: Shift Checker




## Complete Python Source Code

```
"""Check employee shifts for overlaps and short rest periods."""

def parse_time_to_minutes(time_str):
    """
    Converts 24-hour format string 'HH:MM' into integer minutes since midnight.
    Example: '09:30' -> 9 * 60 + 30 = 570
    """
    parts = time_str.strip().split(":")
    hours = int(parts[0])
    minutes = int(parts[1])
    return hours * 60 + minutes


def minutes_to_time_str(total_minutes):
    """Converts integer minutes since midnight back to 'HH:MM' string."""
    hours = total_minutes // 60
    minutes = total_minutes % 60
    return f"{hours:02d}:{minutes:02d}"
```

```
def detect_shift_conflicts(shift_tuples):
    """
    Analyzes list of shift tuples:
    (shift_id, emp_id, emp_name, day, start_str, end_str, dept)
    Detects:
    1. Direct Time Overlaps for the same employee on the same day.
    2. Insufficient Rest (< 480 mins / 8 hours) between shifts on the same day.
    3. Excessive Shift Duration (> 10 hours / 600 mins).
    """
    num_shifts = len(shift_tuples)
    conflicts = []

    # 1. Single Shift Duration Audit
    for shift in shift_tuples:
        s_id, e_id, name, day, s_str, e_str, dept = shift
        s_min = parse_time_to_minutes(s_str)
        e_min = parse_time_to_minutes(e_str)
        duration = e_min - s_min

        if duration <= 0:
            conflicts.append((
                "INVALID_TIMING",
                f"Shift {s_id} for {name} ({e_id}) on {day} has invalid duration (Ends at or before
start: {s_str} - {e_str}).",
                (s_id,)
            ))
        elif duration > 600:  # > 10 hours
            conflicts.append((
                "EXCESSIVE_DURATION",
                f"Shift {s_id} for {name} ({e_id}) on {day} exceeds maximum daily work limit:
{duration//60}h {duration%60}m > 10h limit.",
                (s_id,)
            ))

    # 2. Pairwise Shift Comparison using nested loops
    for i in range(num_shifts):
        s1 = shift_tuples[i]
        id1, emp1, name1, day1, start1_str, end1_str, dept1 = s1
        start1 = parse_time_to_minutes(start1_str)
        end1 = parse_time_to_minutes(end1_str)

        for j in range(i + 1, num_shifts):
            s2 = shift_tuples[j]
            id2, emp2, name2, day2, start2_str, end2_str, dept2 = s2
            start2 = parse_time_to_minutes(start2_str)
            end2 = parse_time_to_minutes(end2_str)

            # Conflicts occur if both shifts belong to the same employee on the same day
            if emp1 == emp2 and day1 == day2:
                # Check Overlap: max(start1, start2) < min(end1, end2)
                overlap_start = start1 if start1 > start2 else start2
                overlap_end = end1 if end1 < end2 else end2

                if overlap_start < overlap_end:
                    overlap_duration = overlap_end - overlap_start
                    conflicts.append((
                        "DIRECT_OVERLAP",
                        f"Collision between {id1} ({start1_str}-{end1_str}) and {id2} ({start2_str}-
{end2_str}) for {name1} ({emp1}) on {day1}. Overlap: {overlap_duration} mins.",
                        (id1, id2)
                    ))
                else:
                    # Check Rest Interval between successive shifts
                    # Case A: Shift 1 ends before Shift 2 starts
```

```
                    if end1 <= start2:
                        rest_gap = start2 - end1
                        if rest_gap < 480:  # Less than 8 hours rest
                            conflicts.append((
                                "INSUFFICIENT_REST",
                                f"Rest violation between {id1} and {id2} for {name1} on {day1}: Rest
gap is only {rest_gap//60}h {rest_gap%60}m (< 8h required).",
                                (id1, id2)
                            ))
                    # Case B: Shift 2 ends before Shift 1 starts
                    elif end2 <= start1:
                        rest_gap = start1 - end2
                        if rest_gap < 480:
                            conflicts.append((
                                "INSUFFICIENT_REST",
                                f"Rest violation between {id2} and {id1} for {name1} on {day1}: Rest
gap is only {rest_gap//60}h {rest_gap%60}m (< 8h required).",
                                (id1, id2)
                            ))

    return conflicts


def display_schedule_audit(shifts, conflicts):
    """Renders formatted shift roster and itemized conflict audit."""
    border = "=" * 94
    dash = "-" * 94

    print("\n" + border)
    print(" " * 22 + "CAMPUS HOSPITAL & FACILITY MANAGEMENT")
    print(" " * 21 + "EMPLOYEE SHIFT SCHEDULE & CONFLICT AUDIT")
    print(border)
    print(f"{'Shift ID':<10}{'Emp ID':<10}{'Employee Name':<18}{'Day':<11}{'Start':<8}{'End':<8}
{'Duration':<11}{'Department':<16}")
    print(dash)

    for s in shifts:
        s_id, e_id, name, day, s_str, e_str, dept = s
        s_min = parse_time_to_minutes(s_str)
        e_min = parse_time_to_minutes(e_str)
        dur_mins = e_min - s_min
        dur_str = f"{dur_mins//60}h {dur_mins%60:02d}m" if dur_mins > 0 else "INVALID"
        print(f"{s_id:<10}{e_id:<10}{name:<18}{day:<11}{s_str:<8}{e_str:<8}{dur_str:<11}{dept:<16}")

    print(dash)
    print(f"Total Shifts Scheduled: {len(shifts)} | Conflicts / Violations Detected:
{len(conflicts)}")
    print(dash)

    if len(conflicts) > 0:
        print("[!] DETECTED SCHEDULE CONFLICTS & LABOUR VIOLATIONS:")
        idx = 1
        for conf in conflicts:
            category, details, involved = conf
            print(f"    [{idx}] CATEGORY: {category}")
            print(f"        Details : {details}")
            print(f"        Shifts  : {', '.join(involved)}")
            idx += 1
    else:
        print("[*] CLEAN SCHEDULE: Zero overlaps, adequate rest intervals, and compliant shift
lengths.")

    print(border + "\n")
```

```
def run_demo():
    """Demonstration shift roster showcasing clean shifts, overlaps, rest violations, and duration
violations."""
    shift_roster = [
        ("SH-101", "EMP-01", "Dr. A. Sharma",   "Monday", "08:00", "14:00", "Emergency"),
        ("SH-102", "EMP-01", "Dr. A. Sharma",   "Monday", "13:00", "19:00", "ICU Support"),       #
Direct Overlap with SH-101 (13:00 < 14:00)
        ("SH-103", "EMP-02", "Nurse B. Devi",   "Monday", "07:00", "15:00", "General Ward"),
        ("SH-104", "EMP-02", "Nurse B. Devi",   "Monday", "19:00", "23:00", "Night Check"),       #
Insufficient Rest (15:00 to 19:00 is 4h < 8h)
        ("SH-105", "EMP-03", "Tech C. Rao",     "Monday", "08:00", "20:00", "Radiology Lab"),     #
12h duration > 10h limit
        ("SH-106", "EMP-04", "Dr. D. Varma",    "Monday", "09:00", "17:00", "Cardiology"),        #
Compliant single shift
        ("SH-107", "EMP-01", "Dr. A. Sharma",   "Tuesday", "09:00", "17:00", "OPD Clinic")        #
Tuesday shift (different day - valid)
    ]

    conflicts = detect_shift_conflicts(shift_roster)
    display_schedule_audit(shift_roster, conflicts)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================================
                      CAMPUS HOSPITAL & FACILITY MANAGEMENT
                     EMPLOYEE SHIFT SCHEDULE & CONFLICT AUDIT
================================================================================================
Shift ID  Emp ID    Employee Name     Day        Start   End     Duration   Department
------------------------------------------------------------------------------------------------
SH-101    EMP-01    Dr. A. Sharma     Monday     08:00   14:00   6h 00m     Emergency
SH-102    EMP-01    Dr. A. Sharma     Monday     13:00   19:00   6h 00m     ICU Support
SH-103    EMP-02    Nurse B. Devi     Monday     07:00   15:00   8h 00m     General Ward
SH-104    EMP-02    Nurse B. Devi     Monday     19:00   23:00   4h 00m     Night Check
SH-105    EMP-03    Tech C. Rao       Monday     08:00   20:00   12h 00m    Radiology Lab
SH-106    EMP-04    Dr. D. Varma      Monday     09:00   17:00   8h 00m     Cardiology
SH-107    EMP-01    Dr. A. Sharma     Tuesday    09:00   17:00   8h 00m     OPD Clinic
------------------------------------------------------------------------------------------------
Total Shifts Scheduled: 7 | Conflicts / Violations Detected: 3
------------------------------------------------------------------------------------------------
[!] DETECTED SCHEDULE CONFLICTS & LABOUR VIOLATIONS:
    [1] CATEGORY: EXCESSIVE_DURATION
        Details : Shift SH-105 for Tech C. Rao (EMP-03) on Monday exceeds maximum daily work limit:
12h 0m > 10h limit.
        Shifts  : SH-105
    [2] CATEGORY: DIRECT_OVERLAP
        Details : Collision between SH-101 (08:00-14:00) and SH-102 (13:00-19:00) for Dr. A. Sharma
(EMP-01) on Monday. Overlap: 60 mins.
        Shifts  : SH-101, SH-102
    [3] CATEGORY: INSUFFICIENT_REST
        Details : Rest violation between SH-103 and SH-104 for Nurse B. Devi on Monday: Rest gap is
only 4h 0m (< 8h required).
        Shifts  : SH-103, SH-104
================================================================================================
```

# Case 56: Student Groups

## What it does

This program sorts students by score and shares them between groups in alternating directions. This helps avoid putting all the highest scores in one group.

## Main steps

Sort students from highest score to lowest.
Create the required number of groups.
Add students from left to right, then right to left.
Compare the group totals and print the groups.

## Python topics

Lists, tuples, loops, sorting, and arithmetic.

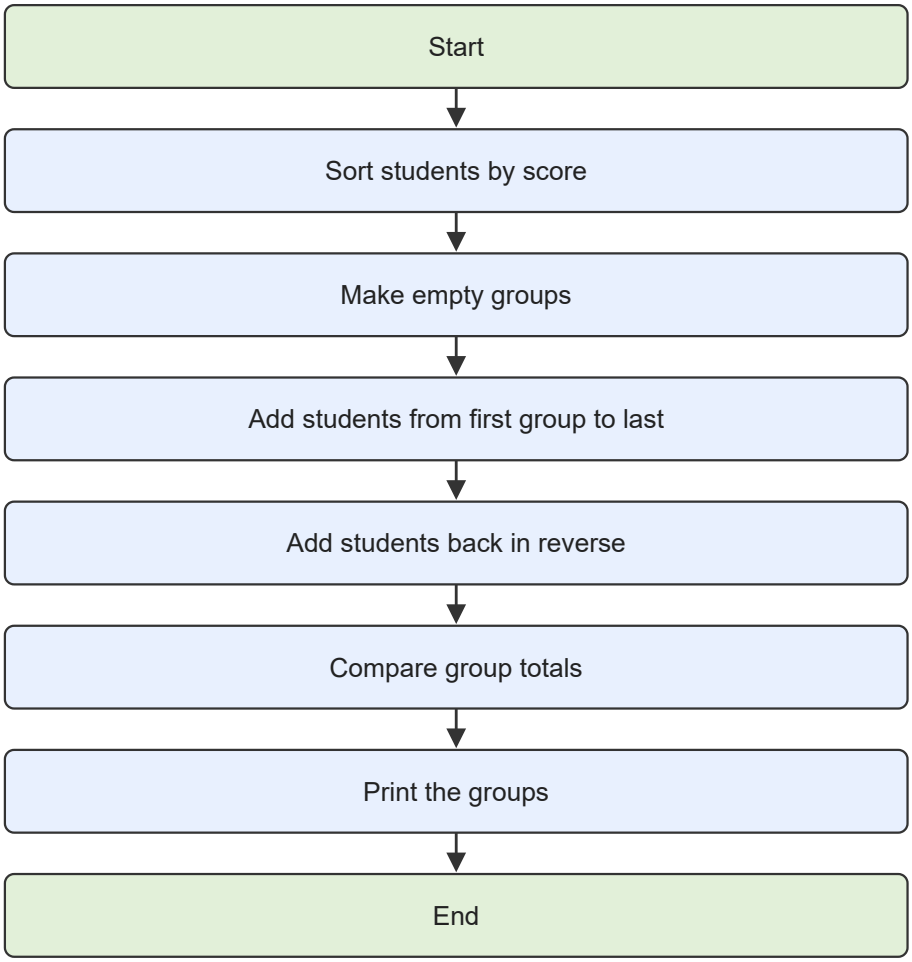
## Run it

```
python solution.py
```

The program uses sample students. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

# Case 56: Student Groups




## Complete Python Source Code

```
"""Sort students by score and share them between groups."""

def bubble_sort_students_descending(student_list):
    """
    Sorts a list of student tuples (std_id, name, score)
    in descending order of score using bubble sort.
    """
    sorted_students = []
    for s in student_list:
        sorted_students.append(s)

    n = len(sorted_students)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            # Unpack scores to compare
            id1, name1, score1 = sorted_students[j]
            id2, name2, score2 = sorted_students[j + 1]
            if score1 < score2:
                # Swap elements
                temp = sorted_students[j]
```

```
                sorted_students[j] = sorted_students[j + 1]
                sorted_students[j + 1] = temp

    return sorted_students


def allocate_serpentine_groups(sorted_students, num_groups):
    """
    Allocates sorted students into num_groups using a Serpentine (Snake Draft) algorithm.
    Round 1: 0, 1, ..., K-1
    Round 2: K-1, K-2, ..., 0
    Round 3: 0, 1, ..., K-1
    This mathematically minimizes the disparity between team total scores.
    """
    groups = []
    for _ in range(num_groups):
        groups.append([])

    current_team_idx = 0
    direction = 1  # 1 for forward, -1 for reverse

    for student in sorted_students:
        groups[current_team_idx].append(student)

        # Advance index based on current serpentine direction
        if direction == 1:
            if current_team_idx == num_groups - 1:
                # Reached the right boundary: reverse direction for next allocation
                direction = -1
            else:
                current_team_idx += 1
        else:
            if current_team_idx == 0:
                # Reached the left boundary: reverse direction for next allocation
                direction = 1
            else:
                current_team_idx -= 1

    return groups


def compute_group_metrics(groups):
    """
    Calculates total score, average score, min, max, and count for each group.
    Returns (group_stats, overall_disparity)
    """
    group_stats = []
    totals = []

    for g_idx in range(len(groups)):
        roster = groups[g_idx]
        total_score = 0.0
        min_score = 1000.0
        max_score = -1.0

        for std in roster:
            _, _, score = std
            total_score += score
            if score < min_score:
                min_score = score
            if score > max_score:
                max_score = score

        count = len(roster)
```

```
        avg_score = round(total_score / count, 2) if count > 0 else 0.0
        totals.append(total_score)

        group_stats.append((
            g_idx + 1,
            count,
            round(total_score, 1),
            avg_score,
            min_score if count > 0 else 0.0,
            max_score if count > 0 else 0.0,
            roster
        ))

    # Overall cohort score spread
    highest_team_total = totals[0]
    lowest_team_total = totals[0]
    for t in totals:
        if t > highest_team_total:
            highest_team_total = t
        if t < lowest_team_total:
            lowest_team_total = t

    disparity = round(highest_team_total - lowest_team_total, 2)
    return group_stats, disparity


def display_balanced_groups_report(group_stats, disparity, num_groups):
    """Prints a structured team allocation report and fairness audit."""
    border = "=" * 90
    dash = "-" * 90

    print("\n" + border)
    print(" " * 22 + "CAMPUS HACKATHON & PROJECT SELECTION")
    print(" " * 20 + "SERPENTINE SCORE-BALANCED GROUP ALLOCATION")
    print(border)
    print(f"Target Number of Teams : {num_groups} Teams")
    print(f"Maximum Total Disparity: {disparity} points between highest and lowest team totals")
    print(dash)

    for stat in group_stats:
        team_num, count, total, avg, min_s, max_s, roster = stat
        print(f"\nTEAM #{team_num:<2} | Members: {count} | Total Score: {total:>6.1f} | Avg Score:
{avg:>5.2f} | Range: [{min_s} - {max_s}]")
        print(f"{'  Roll ID':<12}{'Student Name':<22}{'Score':>8}")
        print("  " + "-" * 42)
        for std in roster:
            s_id, name, score = std
            print(f"  {s_id:<10}{name:<22}{score:>8.1f}")

    print("\n" + dash)
    print("SUMMARY COMPARATIVE SCORE TABLE:")
    print(f"{'Team ID':<10}{'Member Count':>15}{'Total Score':>15}{'Average Score':>15}{'Balance
Status':>20}")
    print(dash)
    for stat in group_stats:
        team_num, count, total, avg, _, _, _ = stat
        status = "WELL-BALANCED" if disparity <= 10.0 else "ACCEPTABLE"
        print(f"Team {team_num:<5}{count:>15}{total:>15.1f}{avg:>15.2f}{status:>20}")
    print(border + "\n")


def run_demo():
    """Demonstrate score balancing on 16 students into 4 balanced teams."""
    students = [
```

```
        ("26CS01", "Aaditya Roy",    98.5),
        ("26CS02", "Bhavya Sri",     95.0),
        ("26CS03", "Charan Teja",    92.0),
        ("26CS04", "Divya Reddy",    89.0),
        ("26CS05", "Eshwar Rao",     86.5),
        ("26CS06", "Fatima Zahra",   84.0),
        ("26CS07", "Gautam Krishna", 81.5),
        ("26CS08", "Harika Devi",    78.0),
        ("26CS09", "Indrajit Paul",  75.5),
        ("26CS10", "Jaya Lakshmi",   72.0),
        ("26CS11", "Kalyan Varma",   69.0),
        ("26CS12", "Lavanya Nair",   66.5),
        ("26CS13", "Manoj Kumar",    63.0),
        ("26CS14", "Nandini Sen",    59.5),
        ("26CS15", "Omkar Nath",     55.0),
        ("26CS16", "Pooja Hegde",    50.0)
    ]

    num_teams = 4
    sorted_std = bubble_sort_students_descending(students)
    groups = allocate_serpentine_groups(sorted_std, num_teams)
    stats, disparity = compute_group_metrics(groups)
    display_balanced_groups_report(stats, disparity, num_teams)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================================
                      CAMPUS HACKATHON & PROJECT SELECTION
                    SERPENTINE SCORE-BALANCED GROUP ALLOCATION
================================================================================================
Target Number of Teams : 4 Teams
Maximum Total Disparity: 3.0 points between highest and lowest team totals
------------------------------------------------------------------------------------------------

TEAM #1  | Members: 4 | Total Score:  302.0 | Avg Score: 75.50 | Range: [50.0 - 98.5]
  Roll ID   Student Name             Score
  ----------------------------------------
  26CS01    Aaditya Roy               98.5
  26CS08    Harika Devi               78.0
  26CS09    Indrajit Paul             75.5
  26CS16    Pooja Hegde               50.0

TEAM #2  | Members: 4 | Total Score:  303.5 | Avg Score: 75.88 | Range: [55.0 - 95.0]
  Roll ID   Student Name             Score
  ----------------------------------------
  26CS02    Bhavya Sri                95.0
  26CS07    Gautam Krishna            81.5
  26CS10    Jaya Lakshmi              72.0
  26CS15    Omkar Nath                55.0

TEAM #3  | Members: 4 | Total Score:  304.5 | Avg Score: 76.12 | Range: [59.5 - 92.0]
  Roll ID   Student Name             Score
  ----------------------------------------
  26CS03    Charan Teja               92.0
  26CS06    Fatima Zahra              84.0
  26CS11    Kalyan Varma              69.0
  26CS14    Nandini Sen               59.5
```

```
TEAM #4  | Members: 4 | Total Score:  305.0 | Avg Score: 76.25 | Range: [63.0 - 89.0]
  Roll ID   Student Name             Score
  ----------------------------------------
  26CS04    Divya Reddy               89.0
  26CS05    Eshwar Rao                86.5
  26CS12    Lavanya Nair              66.5
  26CS13    Manoj Kumar               63.0

--------------------------------------------------------------------------------------------
SUMMARY COMPARATIVE SCORE TABLE:
Team ID      Member Count    Total Score  Average Score      Balance Status
--------------------------------------------------------------------------------------------
Team 1                  4          302.0          75.50       WELL-BALANCED
Team 2                  4          303.5          75.88       WELL-BALANCED
Team 3                  4          304.5          76.12       WELL-BALANCED
Team 4                  4          305.0          76.25       WELL-BALANCED
============================================================================================
```

# Case 75: Student Record Checker

## What it does

This program checks student records for common problems, such as repeated roll numbers, invalid marks, or attendance outside the allowed range.

## Main steps

Go through each student record.
Check the roll number, name, marks, and attendance.
Keep valid records and note the problems in other records.
Print the results.

## Python topics

Strings, tuples, lists, loops, conditions, and range checking.

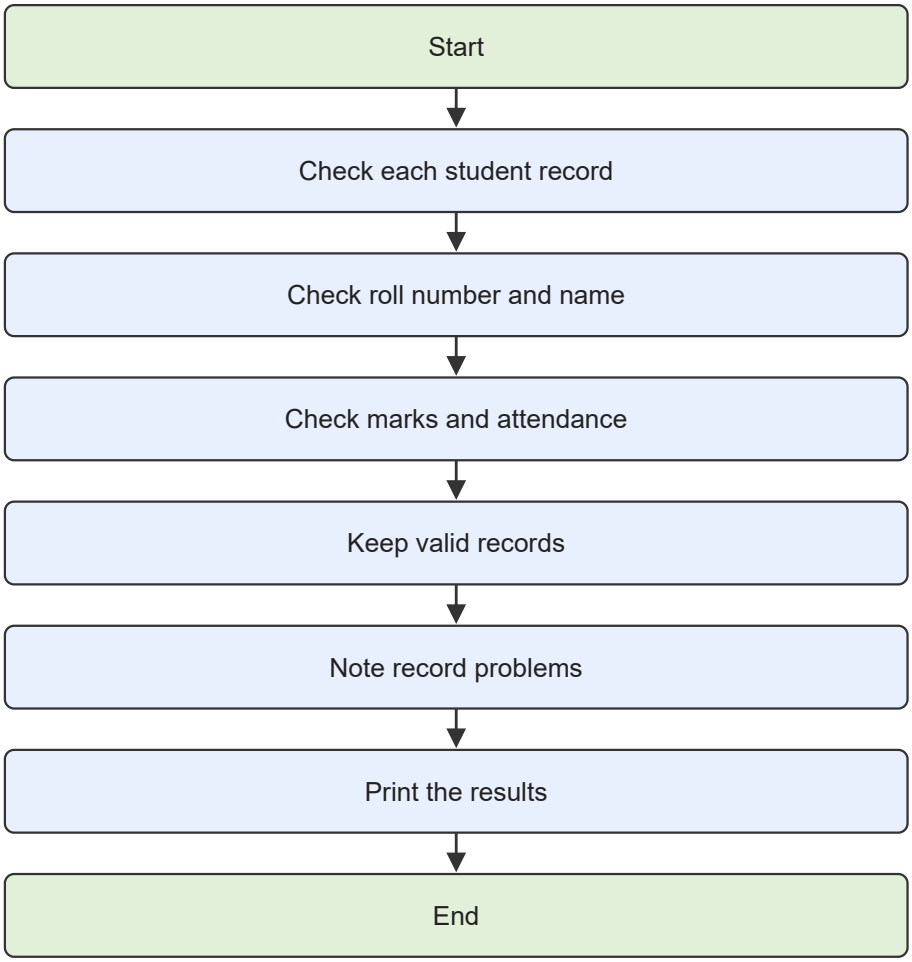
## Run it

```
python solution.py
```

The program uses sample records. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

# Case 75: Student Record Checker




## Complete Python Source Code

```
"""Check student records for common mistakes."""

def validate_roll_number_format(roll_no):
    """
    Validates institutional roll number syntax: e.g., '26CSE101'
    Format: 2 digits (Year) + 3 letters (Branch: CSE/ECE/MEC/INF) + 3 digits (Serial)
    Length must be exactly 8 characters.
    """
    if not isinstance(roll_no, str):
        return False, "Roll number must be a string."
    if len(roll_no) != 8:
        return False, f"Invalid length {len(roll_no)}; roll number must be exactly 8 characters."

    year_part = roll_no[:2]
    branch_part = roll_no[2:5].upper()
    serial_part = roll_no[5:]

    if not (year_part.isdigit() and serial_part.isdigit()):
        return False, "Year prefix (first 2) and serial (last 3) must be numeric digits."
```

```
    valid_branches = ["CSE", "ECE", "MEC", "INF", "CIV", "EEE"]
    if branch_part not in valid_branches:
        return False, f"Unknown department code '{branch_part}'. Must be one of {valid_branches}."

    return True, "Valid"


def validate_student_name(name):
    """Validates student full name: must have at least 2 alphabetic characters."""
    if not isinstance(name, str):
        return False, "Name must be a string."
    cleaned = name.replace(" ", "").replace(".", "")
    if len(cleaned) < 2:
        return False, "Name is too short (less than 2 characters)."
    if not cleaned.isalpha():
        return False, f"Name contains invalid non-alphabetic characters: '{name}'."
    return True, "Valid"


def validate_marks_tuple(marks):
    """Validates tuple of marks: must be non-empty, all numbers within [0.0, 100.0]."""
    if not isinstance(marks, tuple):
        return False, "Marks record must be structured as a tuple."
    if len(marks) == 0:
        return False, "Marks tuple is empty."
    for m in marks:
        if not (isinstance(m, int) or isinstance(m, float)):
            return False, f"Non-numeric mark encountered: {m}"
        if m < 0.0 or m > 100.0:
            return False, f"Mark {m} out of valid bounds [0.0, 100.0]."
    return True, "Valid"


def validate_attendance(att_pct):
    """Validates attendance percentage: must be between 0.0 and 100.0."""
    if not (isinstance(att_pct, int) or isinstance(att_pct, float)):
        return False, "Attendance must be a numeric percentage."
    if att_pct < 0.0 or att_pct > 100.0:
        return False, f"Attendance {att_pct}% out of range [0.0, 100.0]."
    return True, "Valid"


def audit_student_database(records):
    """
    Audits a database list of student tuples:
    (roll_no, name, marks_tuple, attendance_pct, fee_paid, standing)
    Detects duplicate IDs, format errors, value anomalies, and academic contradictions.
    """
    seen_roll_numbers = []
    clean_records = []
    defect_log = []

    for idx in range(len(records)):
        entry = records[idx]
        record_num = idx + 1

        # Verify tuple structure
        if not isinstance(entry, tuple) or len(entry) != 6:
            defect_log.append((record_num, "MALFORMED_RECORD", "Record is not a 6-element tuple.",
str(entry)))
            continue

        roll_no, name, marks, att, fee, standing = entry
        entry_defects = []
```

```
        # 1. Roll number format
        is_roll_ok, roll_msg = validate_roll_number_format(roll_no)
        if not is_roll_ok:
            entry_defects.append(f"Roll Error: {roll_msg}")
        else:
            # Check duplicates
            if roll_no in seen_roll_numbers:
                entry_defects.append(f"DUPLICATE_ID: Roll number '{roll_no}' has already been
registered.")
            else:
                seen_roll_numbers.append(roll_no)

        # 2. Name validation
        is_name_ok, name_msg = validate_student_name(name)
        if not is_name_ok:
            entry_defects.append(f"Name Error: {name_msg}")

        # 3. Marks validation
        is_marks_ok, marks_msg = validate_marks_tuple(marks)
        if not is_marks_ok:
            entry_defects.append(f"Marks Error: {marks_msg}")

        # 4. Attendance validation
        is_att_ok, att_msg = validate_attendance(att)
        if not is_att_ok:
            entry_defects.append(f"Attendance Error: {att_msg}")

        # 5. Fee status validation
        if not isinstance(fee, bool):
            entry_defects.append(f"Fee status must be a boolean True/False (got {fee}).")

        # 6. Academic Standing Contradiction Check
        # If standing == "HONORS", student must have att >= 75%, fee == True, and all marks >= 60.0
        if standing == "HONORS":
            if is_marks_ok:
                min_m = marks[0]
                for m in marks:
                    if m < min_m:
                        min_m = m
                if min_m < 60.0:
                    entry_defects.append(f"Contradiction: Student marked 'HONORS' but has a low mark
({min_m} < 60.0).")
            if is_att_ok and att < 75.0:
                entry_defects.append(f"Contradiction: Student marked 'HONORS' but attendance is
deficient ({att}% < 75%).")

        # Catalog verdict
        if len(entry_defects) > 0:
            defect_log.append((record_num, "INTEGRITY_VIOLATION", "; ".join(entry_defects), roll_no))
        else:
            clean_records.append(entry)

    return clean_records, defect_log


def display_audit_report(total_scanned, clean_records, defect_log):
    """Renders formatted student database consistency report."""
    border = "=" * 94
    dash = "-" * 94

    print("\n" + border)
    print(" " * 22 + "DR. RVR NRI INSTITUTE OF TECHNOLOGY")
    print(" " * 20 + "ACADEMIC DATABASE CONSISTENCY & INTEGRITY AUDIT")
```

```
    print(border)
    print(f"Total Raw Records Ingested : {total_scanned}")
    print(f"Verified Consistent Records: {len(clean_records)}
({round(len(clean_records)/total_scanned*100, 1)}%)")
    print(f"Defective / Flagged Records: {len(defect_log)} ({round(len(defect_log)/total_scanned*100,
1)}%)")
    print(dash)

    print("VERIFIED CLEAN STUDENT RECORDS:")
    print(f"{'Roll No':<11}{'Student Name':<20}{'Marks':<22}{'Att %':>8}{'Fee':>7}{'Standing':>12}")
    print(dash)
    for rec in clean_records:
        r, n, m, a, f, s = rec
        marks_str = str(m)
        fee_str = "PAID" if f else "DUE"
        print(f"{r:<11}{n:<20}{marks_str:<22}{a:>7.1f}%{fee_str:>7}  {s:>12}")

    print(dash)

    if len(defect_log) > 0:
        print("[!] INTEGRITY DEFECT & INCONSISTENCY LOG:")
        for defect in defect_log:
            rec_id, cat, desc, ref = defect
            print(f"    - Record #{rec_id} (Ref: {ref}) | Type: {cat}")
            print(f"      Defects: {desc}")
    else:
        print("[*] Academic database contains zero defects or inconsistencies.")

    print(border + "\n")


def run_demo():
    """Demo database containing clean records, duplicates, bad formats, and contradictions."""
    sample_database = [
        # 1. Clean valid record
        ("26CSE101", "Aaditya Roy", (92.0, 88.0, 95.0), 86.5, True, "HONORS"),

        # 2. Duplicate Roll Number (26CSE101 repeated)
        ("26CSE101", "Bhavya Sri", (80.0, 75.0, 82.0), 90.0, True, "REGULAR"),

        # 3. Invalid Roll format (year non-digit, dept code invalid 'XYZ')
        ("XXXYZ999", "Charan Teja", (65.0, 70.0, 68.0), 78.0, True, "REGULAR"),

        # 4. Out-of-bounds mark (-15.0 and 110.0)
        ("26ECE201", "Divya Reddy", (-15.0, 85.0, 110.0), 82.0, True, "REGULAR"),

        # 5. Invalid attendance (> 100%) and short/numeric name
        ("26MEC301", "E1", (60.0, 65.0, 70.0), 108.5, False, "PROBATION"),

        # 6. Academic Contradiction: Marked HONORS but attendance is only 62.0% (< 75%)
        ("26INF401", "Farhan Akhtar", (88.0, 92.0, 90.0), 62.0, True, "HONORS"),

        # 7. Another clean valid record
        ("26CSE102", "Gautam Krishna", (78.0, 82.0, 79.0), 84.0, True, "REGULAR")
    ]

    clean, defects = audit_student_database(sample_database)
    display_audit_report(len(sample_database), clean, defects)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================================
                      DR. RVR NRI INSTITUTE OF TECHNOLOGY
                    ACADEMIC DATABASE CONSISTENCY & INTEGRITY AUDIT
================================================================================================
Total Raw Records Ingested : 7
Verified Consistent Records: 2 (28.6%)
Defective / Flagged Records: 5 (71.4%)
------------------------------------------------------------------------------------------------
VERIFIED CLEAN STUDENT RECORDS:
Roll No    Student Name        Marks                    Att %    Fee    Standing
------------------------------------------------------------------------------------------------
26CSE101   Aaditya Roy         (92.0, 88.0, 95.0)       86.5%   PAID        HONORS
26CSE102   Gautam Krishna      (78.0, 82.0, 79.0)       84.0%   PAID       REGULAR
------------------------------------------------------------------------------------------------
[!] INTEGRITY DEFECT & INCONSISTENCY LOG:
    - Record #2 (Ref: 26CSE101) | Type: INTEGRITY_VIOLATION
      Defects: DUPLICATE_ID: Roll number '26CSE101' has already been registered.
    - Record #3 (Ref: XXXYZ999) | Type: INTEGRITY_VIOLATION
      Defects: Roll Error: Year prefix (first 2) and serial (last 3) must be numeric digits.
    - Record #4 (Ref: 26ECE201) | Type: INTEGRITY_VIOLATION
      Defects: Marks Error: Mark -15.0 out of valid bounds [0.0, 100.0].
    - Record #5 (Ref: 26MEC301) | Type: INTEGRITY_VIOLATION
      Defects: Name Error: Name contains invalid non-alphabetic characters: 'E1'.; Attendance Error:
Attendance 108.5% out of range [0.0, 100.0].
    - Record #6 (Ref: 26INF401) | Type: INTEGRITY_VIOLATION
      Defects: Contradiction: Student marked 'HONORS' but attendance is deficient (62.0% < 75%).
================================================================================================
```

# Case 91: Registration Checker

## What it does

This program checks registration details stored as text. It splits each line into fields and reports records that do not follow the expected format.

## Main steps

Split a record using the `|` character.
Check the registration number, name, date, category, and status.
Look for duplicate registration numbers.
Print accepted records and error messages.

## Python topics

Strings, `split()`, indexing, lists, tuples, loops, and conditions.

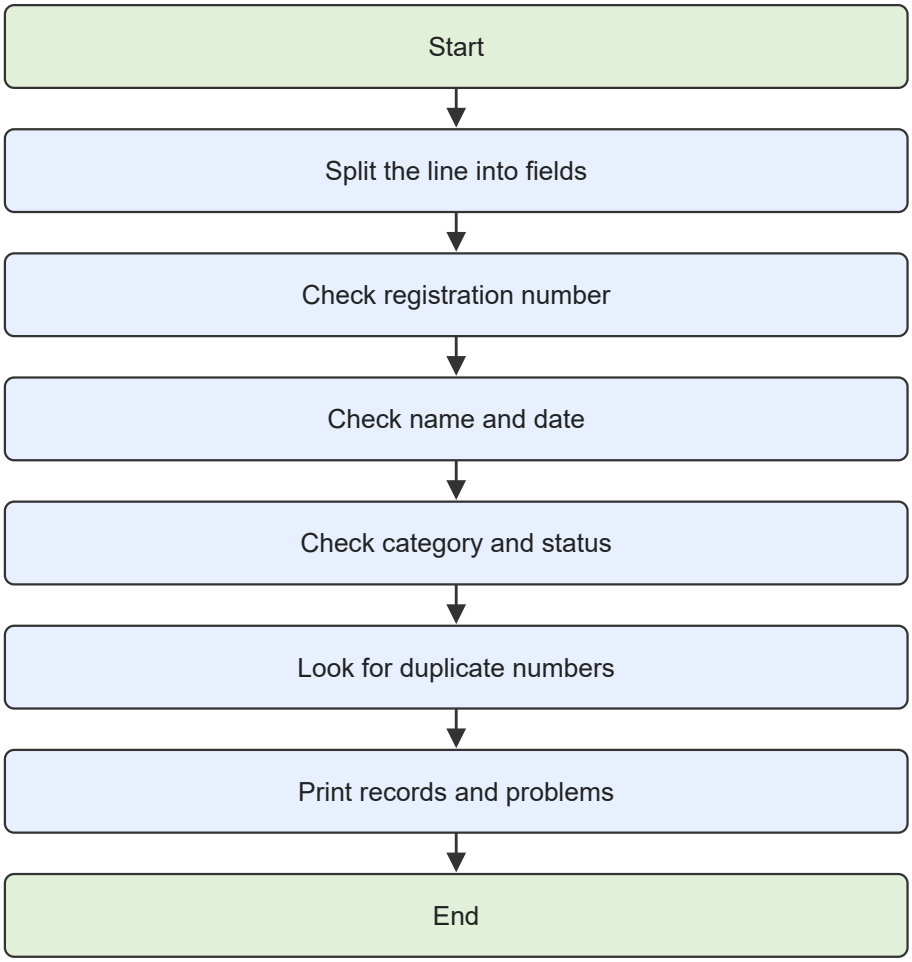
## Run it

```
python solution.py
```

The program uses sample registration strings. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

# Case 91: Registration Checker




## Complete Python Source Code

```
"""Read registration details from text and check their format."""

def parse_and_validate_date(dob_str):
    """
    Validates DOB string in format YYYY-MM-DD.
    Returns (is_valid, error_msg).
    """
    parts = dob_str.split("-")
    if len(parts) != 3:
        return False, f"Malformed date structure '{dob_str}'. Expected YYYY-MM-DD."

    y_str, m_str, d_str = parts
    if not (y_str.isdigit() and m_str.isdigit() and d_str.isdigit()):
        return False, f"Non-numeric date component in '{dob_str}'."

    year = int(y_str)
    month = int(m_str)
    day = int(d_str)

    if year < 1995 or year > 2012:
```

```
        return False, f"Birth year {year} outside valid student age bracket [1995 - 2012]."
    if month < 1 or month > 12:
        return False, f"Month {month} out of valid range [1 - 12]."
    if day < 1 or day > 31:
        return False, f"Day {day} out of valid range [1 - 31]."

    return True, "Valid"


def parse_registration_token(token):
    """
    Parses 'REG#YYYY-DEPT-XXXX'
    Returns (is_valid, error_msg, year, dept, seq).
    """
    if not token.startswith("REG#"):
        return False, "Missing 'REG#' prefix", "", "", ""

    body = token[4:]  # Slice off 'REG#'
    segments = body.split("-")
    if len(segments) != 3:
        return False, f"Registration identifier '{body}' must have 3 hyphen-separated segments (YYYY-
DEPT-XXXX)", "", "", ""

    year_str, dept_str, seq_str = segments
    if not (year_str.isdigit() and len(year_str) == 4):
        return False, f"Invalid admission year '{year_str}'. Expected 4 digits.", "", "", ""
    if not (seq_str.isdigit() and len(seq_str) == 4):
        return False, f"Invalid sequence serial '{seq_str}'. Expected 4 digits.", "", "", ""

    valid_depts = ["CSE", "ECE", "MEC", "INF", "CIV", "EEE"]
    dept_upper = dept_str.upper()
    if dept_upper not in valid_depts:
        return False, f"Invalid department code '{dept_str}'. Must be one of {valid_depts}.", "", "",
""

    return True, "Valid", year_str, dept_upper, seq_str


def analyze_raw_registration_string(raw_line):
    """
    Parses raw delimited string:
    'REG#YYYY-DEPT-XXXX|NAME:Aditya_Sharma|DOB:YYYY-MM-DD|CATEGORY:GENERAL|STATUS:CONFIRMED'
    Returns (is_valid, result_tuple_or_error_tuple).
    """
    cleaned = raw_line.strip()
    if len(cleaned) == 0:
        return False, ("EMPTY_RECORD", "Line is completely empty", raw_line)

    tokens = cleaned.split("|")
    if len(tokens) != 5:
        return False, ("DELIMITER_ERROR", f"Expected 5 pipe-delimited fields, found {len(tokens)}",
raw_line)

    t_reg, t_name, t_dob, t_cat, t_stat = tokens

    # 1. Registration ID validation
    is_reg_ok, reg_msg, year, dept, seq = parse_registration_token(t_reg.strip())
    if not is_reg_ok:
        return False, ("REG_ID_ERROR", reg_msg, raw_line)

    # 2. Name token validation
    if not t_name.strip().startswith("NAME:"):
        return False, ("NAME_FIELD_ERROR", "Field 2 must start with 'NAME:'", raw_line)
    raw_name = t_name.strip()[5:]
```

```
    if len(raw_name) < 2:
        return False, ("NAME_SYNTAX_ERROR", "Name cannot be empty or less than 2 characters",
raw_line)
    # Check that name only contains letters and underscores
    for ch in raw_name:
        if not (ch.isalpha() or ch == '_' or ch == ' '):
            return False, ("NAME_SYNTAX_ERROR", f"Illegal character '{ch}' in name field", raw_line)
    clean_name = raw_name.replace("_", " ")

    # 3. DOB token validation
    if not t_dob.strip().startswith("DOB:"):
        return False, ("DOB_FIELD_ERROR", "Field 3 must start with 'DOB:'", raw_line)
    dob_val = t_dob.strip()[4:]
    is_dob_ok, dob_msg = parse_and_validate_date(dob_val)
    if not is_dob_ok:
        return False, ("DATE_ERROR", dob_msg, raw_line)

    # 4. Category token validation
    if not t_cat.strip().startswith("CATEGORY:"):
        return False, ("CAT_FIELD_ERROR", "Field 4 must start with 'CATEGORY:'", raw_line)
    cat_val = t_cat.strip()[9:].upper()
    valid_categories = ["GENERAL", "OBC", "SC", "ST", "EWS"]
    if cat_val not in valid_categories:
        return False, ("CATEGORY_ENUM_ERROR", f"Unknown category '{cat_val}'. Allowed:
{valid_categories}", raw_line)

    # 5. Status token validation
    if not t_stat.strip().startswith("STATUS:"):
        return False, ("STATUS_FIELD_ERROR", "Field 5 must start with 'STATUS:'", raw_line)
    stat_val = t_stat.strip()[7:].upper()
    valid_statuses = ["CONFIRMED", "PROVISIONAL", "WAITLISTED"]
    if stat_val not in valid_statuses:
        return False, ("STATUS_ENUM_ERROR", f"Unknown status '{stat_val}'. Allowed: {valid_statuses}",
raw_line)

    full_reg_id = f"REG#{year}-{dept}-{seq}"
    parsed_record = (full_reg_id, year, dept, seq, clean_name, dob_val, cat_val, stat_val)
    return True, parsed_record


def process_registration_batch(raw_records):
    """
    Processes a batch of raw registration strings,
    extracts valid tuples, checks duplicate IDs, and categorizes errors.
    """
    valid_parsed_records = []
    registered_ids = []
    error_records = []

    for idx in range(len(raw_records)):
        line = raw_records[idx]
        rec_num = idx + 1
        is_valid, result = analyze_raw_registration_string(line)

        if is_valid:
            reg_id, year, dept, seq, name, dob, cat, stat = result
            # Check for duplicate registration keys
            if reg_id in registered_ids:
                error_records.append((rec_num, "DUPLICATE_KEY_ERROR", f"Duplicate registration
identifier '{reg_id}' detected.", line))
            else:
                registered_ids.append(reg_id)
                valid_parsed_records.append(result)
        else:
```

```
            err_type, err_desc, orig_str = result
            error_records.append((rec_num, err_type, err_desc, orig_str))

    return valid_parsed_records, error_records


def display_registration_audit(valid_records, error_records):
    """Prints a formatted report of parsed structured records and error classifications."""
    border = "=" * 98
    dash = "-" * 98

    print("\n" + border)
    print(" " * 24 + "DR. RVR NRI INSTITUTE OF TECHNOLOGY")
    print(" " * 22 + "CAMPUS ADMISSION REGISTRATION STRING ANALYZER")
    print(border)
    print(f"Total Strings Processed: {len(valid_records) + len(error_records)}")
    print(f"Successfully Extracted : {len(valid_records)}")
    print(f"Rejected / Malformed   : {len(error_records)}")
    print(dash)

    print("PARSED STRUCTURED REGISTRATION DATABASE:")
    print(f"{'Reg ID':<19}{'Dept':<6}{'Year':<6}{'Candidate Name':<22}{'DOB':<12}{'Category':<10}
{'Status':<12}")
    print(dash)
    for rec in valid_records:
        rid, yr, dept, seq, name, dob, cat, stat = rec
        print(f"{rid:<19}{dept:<6}{yr:<6}{name:<22}{dob:<12}{cat:<10}{stat:<12}")

    print(dash)

    if len(error_records) > 0:
        print("[!] PARSING DEFECT CLASSIFICATION & AUDIT LOG:")
        for err in error_records:
            rnum, etype, edesc, raw = err
            print(f"    - Entry #{rnum} | Defect Class: {etype:<20}")
            print(f"      Reason: {edesc}")
            print(f"      Raw   : \"{raw}\"")
    else:
        print("[*] All input registration strings conformed perfectly to specification.")
    print(border + "\n")


def run_demo():
    """Demonstration dataset featuring valid records, delimiter errors, bad dates, and duplicates."""
    raw_inputs = [
        # 1. Perfectly formatted valid record
        "REG#2026-CSE-0142|NAME:Aditya_Sharma|DOB:2006-05-14|CATEGORY:GENERAL|STATUS:CONFIRMED",

        # 2. Valid record for ECE
        "REG#2026-ECE-0205|NAME:Bhavya_Sri_Reddy|DOB:2005-11-20|CATEGORY:OBC|STATUS:PROVISIONAL",

        # 3. Delimiter Error (only 3 fields, missing status and category)
        "REG#2026-CSE-0143|NAME:Charan_Teja|DOB:2006-02-10",

        # 4. Invalid Department code in Reg Token ('BIOTECH' is not in allowed departments)
        "REG#2026-BIO-0999|NAME:Divya_Rao|DOB:2005-08-15|CATEGORY:SC|STATUS:CONFIRMED",

        # 5. Invalid DOB (Month 14 out of range)
        "REG#2026-INF-0410|NAME:Eshwar_Prasad|DOB:2006-14-05|CATEGORY:ST|STATUS:CONFIRMED",

        # 6. Duplicate Key Error (Repeat of REG#2026-CSE-0142)
        "REG#2026-CSE-0142|NAME:Another_Student|DOB:2006-01-01|CATEGORY:GENERAL|STATUS:WAITLISTED",

        # 7. Another valid record
```

```
        "REG#2026-MEC-0315|NAME:Fatima_Zahra|DOB:2006-09-30|CATEGORY:EWS|STATUS:CONFIRMED"
    ]

    valid, errors = process_registration_batch(raw_inputs)
    display_registration_audit(valid, errors)


if __name__ == "__main__":
    run_demo()
```

## Verified Console Output / Execution Capture

```
================================================================================================================
                        DR. RVR NRI INSTITUTE OF TECHNOLOGY
                      CAMPUS ADMISSION REGISTRATION STRING ANALYZER
================================================================================================================
Total Strings Processed: 7
Successfully Extracted : 3
Rejected / Malformed   : 4
----------------------------------------------------------------------------------------------------------------
PARSED STRUCTURED REGISTRATION DATABASE:
Reg ID             Dept  Year  Candidate Name        DOB         Category  Status
----------------------------------------------------------------------------------------------------------------
REG#2026-CSE-0142  CSE   2026  Aditya Sharma         2006-05-14  GENERAL   CONFIRMED
REG#2026-ECE-0205  ECE   2026  Bhavya Sri Reddy      2005-11-20  OBC       PROVISIONAL
REG#2026-MEC-0315  MEC   2026  Fatima Zahra          2006-09-30  EWS       CONFIRMED
----------------------------------------------------------------------------------------------------------------
[!] PARSING DEFECT CLASSIFICATION & AUDIT LOG:
    - Entry #3 | Defect Class: DELIMITER_ERROR
      Reason: Expected 5 pipe-delimited fields, found 3
      Raw   : "REG#2026-CSE-0143|NAME:Charan_Teja|DOB:2006-02-10"
    - Entry #4 | Defect Class: REG_ID_ERROR
      Reason: Invalid department code 'BIO'. Must be one of ['CSE', 'ECE', 'MEC', 'INF', 'CIV',
'EEE'].
      Raw   : "REG#2026-BIO-0999|NAME:Divya_Rao|DOB:2005-08-15|CATEGORY:SC|STATUS:CONFIRMED"
    - Entry #5 | Defect Class: DATE_ERROR
      Reason: Month 14 out of valid range [1 - 12].
      Raw   : "REG#2026-INF-0410|NAME:Eshwar_Prasad|DOB:2006-14-05|CATEGORY:ST|STATUS:CONFIRMED"
    - Entry #6 | Defect Class: DUPLICATE_KEY_ERROR
      Reason: Duplicate registration identifier 'REG#2026-CSE-0142' detected.
      Raw   : "REG#2026-CSE-0142|NAME:Another_Student|DOB:2006-01-
01|CATEGORY:GENERAL|STATUS:WAITLISTED"
================================================================================================================
```

# Case 100: Campus Management

## What it does

This program brings together a few campus tasks. It checks student marks and exam eligibility, registers students for events, and books rooms.

## Main steps

Make sample student, event, and room data.
Calculate student totals, grades, and eligibility.
Try event registrations and check event capacity.
Try room bookings and check for time clashes.
Print the results.

## Python topics

Lists, nested lists, tuples, loops, functions, strings, and conditions.

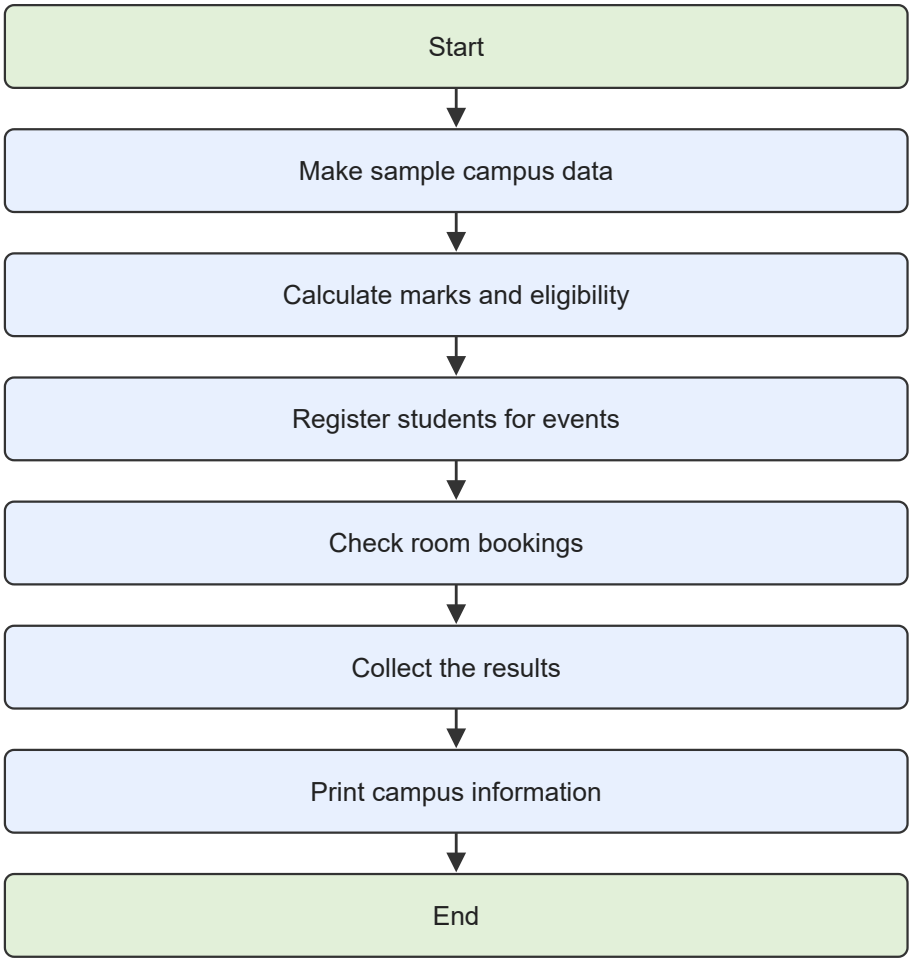
## Run it

```
python solution.py
```

This case uses the same basic ideas as the other programs. See [output.txt](output.txt) and [flowchart.svg](flowchart.svg).

## Visual Program Flowchart Diagram

# Case 100: Campus Management




## Complete Python Source Code

```
"""Show sample student, event, and room information for a campus."""

def create_initial_campus_data():
    """
    Initializes mock database using lists and tuples (strictly adhering to syllabus).
    """
    # Student records: list of tuples
    # (roll_no, name, dept, [marks_sub1, sub2, sub3], attendance_pct, fee_paid)
    students = [
        ("26CSE001", "Aaditya Roy",    "CSE", [92.0, 88.0, 95.0], 88.0, True),
        ("26CSE002", "Bhavya Sri",     "CSE", [78.0, 72.0, 85.0], 82.0, True),
        ("26ECE001", "Charan Teja",    "ECE", [64.0, 58.0, 62.0], 68.5, False),  # Low attendance &
fee due
        ("26ECE002", "Divya Reddy",    "ECE", [85.0, 90.0, 88.0], 91.0, True),
        ("26MEC001", "Eshwar Rao",     "MEC", [52.0, 48.0, 55.0], 71.0, True),   # Low attendance
        ("26INF001", "Fatima Zahra",   "INF", [88.0, 92.0, 90.0], 95.0, True)
    ]

    # Campus Events: list of tuples
    # (event_code, event_name, max_capacity, registered_rolls_list)
```

```
    events = [
        ("EVT-AI", "National AI & Robotics Symposium", 3, ["26CSE001", "26CSE002"]),
        ("EVT-HACK", "48-Hour Dussehra Hackathon", 2, ["26INF001"]),
        ("EVT-WKS", "Cloud Computing Workshop", 4, ["26ECE002", "26MEC001"])
    ]

    # Campus Labs/Venues: list of tuples
    # (venue_id, venue_name, capacity, bookings_list_of_tuples)
    # bookings_list: [(slot_str, booked_by_dept)]
    resources = [
        ("LAB-301", "Advanced Computing Lab", 60, [("09:00-11:00", "CSE"), ("14:00-16:00", "INF")]),
        ("AUD-101", "Sir C.V. Raman Auditorium", 250, [("10:00-13:00", "INSTITUTE")]),
        ("SEM-201", "ECE Seminar Hall", 80, [])
    ]

    return students, events, resources


def compute_student_academics(students):
    """
    Computes total marks, percentage, and grade for each student.
    Returns list of tuples: (roll, name, dept, total, pct, grade, att, fee_status, exam_eligible)
    """
    academic_report = []
    for s in students:
        roll, name, dept, marks, att, fee = s

        # Calculate sum and percentage
        total = 0.0
        for m in marks:
            total += m
        pct = round(total / len(marks), 2)

        # Grade computation
        if pct >= 90.0:
            grade = "O"
        elif pct >= 80.0:
            grade = "A+"
        elif pct >= 70.0:
            grade = "A"
        elif pct >= 60.0:
            grade = "B+"
        elif pct >= 50.0:
            grade = "B"
        elif pct >= 40.0:
            grade = "C"
        else:
            grade = "F"

        # Eligibility criteria: attendance >= 75% AND fee paid
        is_eligible = (att >= 75.0) and fee
        eligibility_str = "ELIGIBLE" if is_eligible else "BARRED"
        fee_str = "PAID" if fee else "DUE"

        academic_report.append((roll, name, dept, total, pct, grade, att, fee_str, eligibility_str))

    return academic_report


def display_academic_dashboard(academic_report):
    """Prints formatted student academic and examination eligibility table."""
    border = "=" * 94
    dash = "-" * 94
    print("\n" + border)
```

```
    print(" " * 26 + "STUDENT ACADEMIC & ELIGIBILITY REPORT")
    print(border)
    print(f"{'Roll No':<11}{'Student Name':<18}{'Dept':<6}{'Total':>8}{'Pct %':>8}{'Grade':>7}{'Att
%':>8}{'Fee':>7}{'Exam Status':>14}")
    print(dash)

    for row in academic_report:
        roll, name, dept, total, pct, grade, att, fee, status = row
        print(f"{roll:<11}{name:<18}{dept:<6}{total:>8.1f}{pct:>7.1f}%{grade:>6}{att:>7.1f}%{fee:>7}
{status:>12}")

    print(dash)

    # Audit alerts
    barred_students = []
    for row in academic_report:
        if row[8] == "BARRED":
            barred_students.append(row)

    if len(barred_students) > 0:
        print("[!] EXAMINATION ADMIT CARD WITHHELD (BARRED DUE TO DEFICIENCIES):")
        for b in barred_students:
            reasons = []
            if b[6] < 75.0:
                reasons.append(f"Shortage of attendance ({b[6]:.1f}% < 75.0%)")
            if b[7] == "DUE":
                reasons.append("Semester tuition fee due")
            print(f"    - {b[1]} ({b[0]} - {b[2]}): {', '.join(reasons)}")
    else:
        print("[*] All students meet academic, attendance, and fee clearance standards.")
    print(border + "\n")


def register_student_for_event(events, event_code, roll_no):
    """
    Registers a student for a campus event respecting maximum capacity limits.
    """
    for e in events:
        code, name, max_cap, reg_list = e
        if code == event_code:
            if roll_no in reg_list:
                return False, f"Student {roll_no} is already registered for {name}."
            if len(reg_list) >= max_cap:
                return False, f"Registration failed! {name} has reached full capacity
({max_cap}/{max_cap})."
            reg_list.append(roll_no)
            return True, f"Success! {roll_no} registered for {name} ({len(reg_list)}/{max_cap}
filled)."
    return False, f"Event code '{event_code}' not found."


def allocate_resource_slot(resources, venue_id, req_slot, dept):
    """
    Allocates a campus venue/lab for a department slot, detecting slot conflicts.
    """
    for res in resources:
        v_id, name, cap, bookings = res
        if v_id == venue_id:
            for b in bookings:
                slot, b_dept = b
                if slot == req_slot:
                    return False, f"Collision! {name} is already booked for slot '{req_slot}' by
{b_dept} department."
            bookings.append((req_slot, dept))
```

```
            return True, f"Success! {name} reserved for {dept} during slot '{req_slot}'."
    return False, f"Venue ID '{venue_id}' not found."


def display_events_and_resources(events, resources):
    """Displays status of campus events and allocated laboratory resources."""
    dash = "-" * 88
    print("\n" + "=" * 88)
    print(" " * 24 + "CAMPUS EVENTS & CAPACITY DASHBOARD")
    print("=" * 88)
    print(f"{'Code':<11}{'Event Name':<38}{'Capacity':<12}{'Enrolled Roll Numbers'}")
    print(dash)
    for e in events:
        code, name, cap, reg_list = e
        enrolled_str = ", ".join(reg_list) if len(reg_list) > 0 else "[Empty]"
        cap_str = f"{len(reg_list)}/{cap}"
        print(f"{code:<11}{name:<38}{cap_str:<12}{enrolled_str}")
    print(dash)

    print("\n" + "=" * 88)
    print(" " * 22 + "CAMPUS VENUE & LAB RESOURCE SCHEDULE")
    print("=" * 88)
    print(f"{'Venue ID':<11}{'Venue Name':<32}{'Cap':>6}{'Active Bookings (Slot -> Dept)'}")
    print(dash)
    for r in resources:
        vid, vname, cap, bookings = r
        b_str = "; ".join([f"{slot} ({dept})" for slot, dept in bookings]) if len(bookings) > 0 else "
[Available All Day]"
        print(f"{vid:<11}{vname:<32}{cap:>6}   {b_str}")
    print("=" * 88 + "\n")


def run_comprehensive_simulation():
    """Runs a complete end-to-end automated simulation test."""
    print("********************************************************************************")
    print("   DR. RVR NRI INSTITUTE OF TECHNOLOGY - SMART CAMPUS MANAGEMENT SIMULATION")
    print("********************************************************************************")

    # 1. Initialize data
    students, events, resources = create_initial_campus_data()

    # 2. Process academics
    academic_report = compute_student_academics(students)
    display_academic_dashboard(academic_report)

    # 3. Test Event Registration Logic
    print("[*] TESTING EVENT REGISTRATION SYSTEM:")
    # Test valid registration
    ok, msg = register_student_for_event(events, "EVT-HACK", "26CSE001")
    print(f"    - Attempt 1: {msg}")

    # Test capacity overflow (EVT-HACK capacity was 2, now full)
    ok, msg = register_student_for_event(events, "EVT-HACK", "26ECE001")
    print(f"    - Attempt 2 (Overflow Test): {msg}")

    # Test duplicate registration
    ok, msg = register_student_for_event(events, "EVT-AI", "26CSE001")
    print(f"    - Attempt 3 (Duplicate Test): {msg}")

    # 4. Test Resource Allocation Logic
    print("\n[*] TESTING LAB & VENUE RESOURCE SCHEDULER:")
    # Test booking collision on LAB-301 (already booked for 09:00-11:00)
    ok, msg = allocate_resource_slot(resources, "LAB-301", "09:00-11:00", "ECE")
    print(f"    - Booking Attempt 1 (Conflict Test): {msg}")
```

```
    # Test successful booking on open slot
    ok, msg = allocate_resource_slot(resources, "SEM-201", "11:00-13:00", "CSE")
    print(f"    - Booking Attempt 2 (Valid Slot)   : {msg}")

    # 5. Display consolidated final campus status
    display_events_and_resources(events, resources)
    print("********************************************************************************")
    print("              SIMULATION COMPLETED SUCCESSFULLY WITH ZERO ERRORS                 ")
    print("********************************************************************************\n")


if __name__ == "__main__":
    run_comprehensive_simulation()
```

## Verified Console Output / Execution Capture

```
********************************************************************************
   DR. RVR NRI INSTITUTE OF TECHNOLOGY - SMART CAMPUS MANAGEMENT SIMULATION
********************************************************************************

================================================================================================
                          STUDENT ACADEMIC & ELIGIBILITY REPORT
================================================================================================
Roll No    Student Name      Dept     Total   Pct %  Grade   Att %    Fee   Exam Status
------------------------------------------------------------------------------------------------
26CSE001   Aaditya Roy       CSE      275.0   91.7%     O   88.0%   PAID      ELIGIBLE
26CSE002   Bhavya Sri        CSE      235.0   78.3%     A   82.0%   PAID      ELIGIBLE
26ECE001   Charan Teja       ECE      184.0   61.3%    B+   68.5%    DUE        BARRED
26ECE002   Divya Reddy       ECE      263.0   87.7%    A+   91.0%   PAID      ELIGIBLE
26MEC001   Eshwar Rao        MEC      155.0   51.7%     B   71.0%   PAID        BARRED
26INF001   Fatima Zahra      INF      270.0   90.0%     O   95.0%   PAID      ELIGIBLE
------------------------------------------------------------------------------------------------
[!] EXAMINATION ADMIT CARD WITHHELD (BARRED DUE TO DEFICIENCIES):
    - Charan Teja (26ECE001 - ECE): Shortage of attendance (68.5% < 75.0%), Semester tuition fee due
    - Eshwar Rao (26MEC001 - MEC): Shortage of attendance (71.0% < 75.0%)
================================================================================================

[*] TESTING EVENT REGISTRATION SYSTEM:
    - Attempt 1: Success! 26CSE001 registered for 48-Hour Dussehra Hackathon (2/2 filled).
    - Attempt 2 (Overflow Test): Registration failed! 48-Hour Dussehra Hackathon has reached full
capacity (2/2).
    - Attempt 3 (Duplicate Test): Student 26CSE001 is already registered for National AI & Robotics
Symposium.

[*] TESTING LAB & VENUE RESOURCE SCHEDULER:
    - Booking Attempt 1 (Conflict Test): Collision! Advanced Computing Lab is already booked for slot
'09:00-11:00' by CSE department.
    - Booking Attempt 2 (Valid Slot)   : Success! ECE Seminar Hall reserved for CSE during slot
'11:00-13:00'.

================================================================================
                       CAMPUS EVENTS & CAPACITY DASHBOARD
================================================================================
Code       Event Name                          Capacity    Enrolled Roll Numbers
--------------------------------------------------------------------------------
EVT-AI     National AI & Robotics Symposium      2/3         26CSE001, 26CSE002
EVT-HACK   48-Hour Dussehra Hackathon            2/2         26INF001, 26CSE001
EVT-WKS    Cloud Computing Workshop              2/4         26ECE002, 26MEC001
--------------------------------------------------------------------------------

================================================================================
```

```
                      CAMPUS VENUE & LAB RESOURCE SCHEDULE
================================================================================================
Venue ID   Venue Name                          CapActive Bookings (Slot -> Dept)
------------------------------------------------------------------------------------------------
LAB-301    Advanced Computing Lab               60   09:00-11:00 (CSE); 14:00-16:00 (INF)
AUD-101    Sir C.V. Raman Auditorium           250   10:00-13:00 (INSTITUTE)
SEM-201    ECE Seminar Hall                     80   11:00-13:00 (CSE)
================================================================================================

********************************************************************************
              SIMULATION COMPLETED SUCCESSFULLY WITH ZERO ERRORS
********************************************************************************
```